Numerical Methods in Soft Matter

Sofia Salvatore

1 Lecture 1

1.1 Exercise 1.1

The equation of the ellipsoid is

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1 \tag{1}$$

the analytical volume is V $= \frac{4}{3}\pi abc$. The volume is numerically estimated with hit and miss method by generating 3 random numbers x in $[a, a]$, y in $[b, b]$ and z in $[c, c]$. If x, y, z, with $a = 3, b = 2, c = 2$ generated realize condition 1, then we have a hit, otherwise a miss. The volume then is

$$V = V_{box}\frac{\#\text{hit}}{\#\text{tries}} \tag{2}$$

where V_{box} is the volume $8a^2b^2c^2$ of the box containing the ellipsoid. The same box is used for both ellipsoids (first ellipsoid: $a = 3, b = 2, c = 2$ and second ellipsoid $a_2 = 3, b_2 = 1, c_2 = 1$). The relative error between estimated volume and analytical volume is thus computed as $\delta = \frac{V - V_{th}}{V_{th}}$. As one can see in figure 1, the choice of not changing the size of the box for the second ellipsoid results in a slower convergence (red line), compared to the result of the first ellipsoid (blue line). In fact, if now the sampling for the second ellipsoid is done using another box $[a_2, a_2]x[b_2, b_2]x[c_2, c_2]$, better suited for this case, the convergence to the analytical value improves (see green line): the method is definitely more efficient.

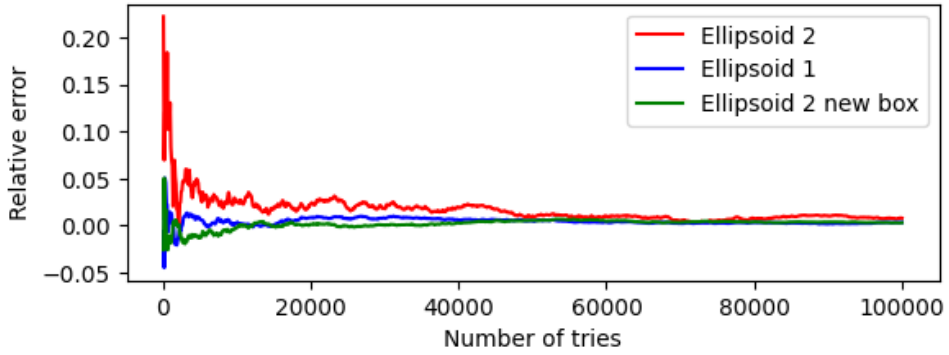

Figure 1: Relative error convergence to analytical value as a function of the number of tries

1.2 Exercise 1.2

With probability density function $\rho_1(x) = c\,x\,e^{-x^2}$ defined in $[0, \infty)$ the normalization is $\int_0^\infty c\,x\,e^{-x^2}\,dx = 1$ from which $c = 2$ is derived. Therefore $\rho_1(x) = 2\,x\,e^{-x^2}$.
In order to sample numbers with the above distribution it is necessary to first compute the associated Cumulative distribution function $F_1(x)$: $F_1(x) = \int_0^x 2\,x\,e^{-x^2}\,dx = 1 - e^{-x^2}$. The inversion method then states that if a random number ξ is generated between [0,1], then $x = F_1^{-1}(\xi)$ is distributed according to $\rho_1(x)$. In this case, $F_1^{-1}(\xi) = \sqrt{-\log(1-\xi)}$. The same calculations are performed for the second probability density function $\rho_2(x) = b\,x^4$, defined in $[0, 3]$. From the normalization $\int_0^3 b\,x^4\,dx = 1$ one obtains that $b = \frac{5}{3^5}$. The cumulative distribution function is $F_2(x) = \int_0^x \frac{5}{3^5}\,x^4\,dx$. Having as before $\xi \in [0, 1]$, then $x = (3^5\,\xi)^{1/5}$ is distributed according to $\rho_2(x)$. Histograms are plotted in figure 2

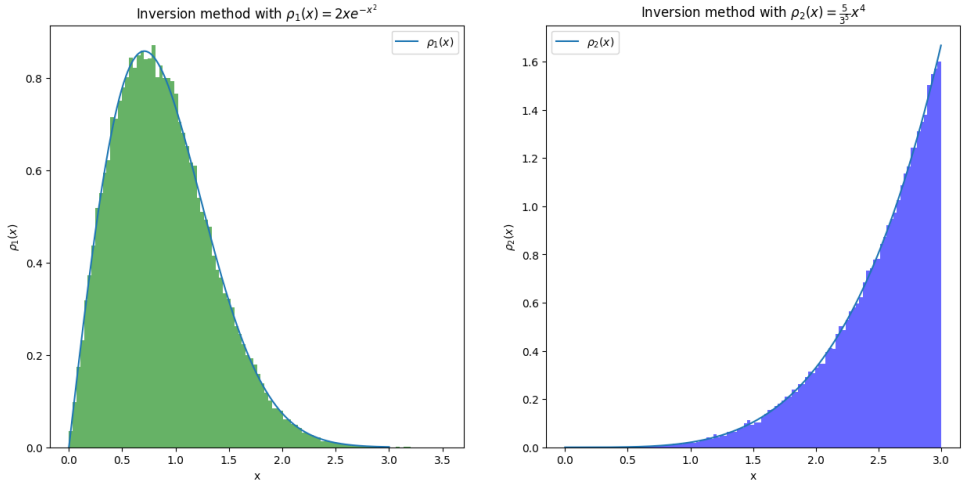

Figure 2

2 Lecture 2

2.1 Exercise 2.1

Normalization constant A_f is $A_f = \frac{\sqrt{2}}{eK_{1/4}(1)} \approx 1.2078$. The normalized PDF is given by

$$f(x) = \frac{\sqrt{2}}{eK_{1/4}(1)e^{-8(\frac{x^2}{2}+\frac{x^4}{4})}} \tag{3}$$

Now the rejection method is applied. A new proper distribution is $g(x)$ chosen from which is easier to sample random number and such that $f(x) < cg(x)$ where c is greater and usually close to 1. For this case the choice is the normal distribution $g(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{\frac{-(x-\mu)^2}{2\sigma^2}}$ with $\mu = 0$ and $\sigma = 1$. So a number η is generated using `np.random.normal(0,1)`. Then the algorithm of rejection method is applied: generated ξ, a random number in [0,1], if $\xi < f(\eta)/(cg(\eta))$, with $c = 3$, η is accepted, otherwise is rejected. Result is shown in figure 3

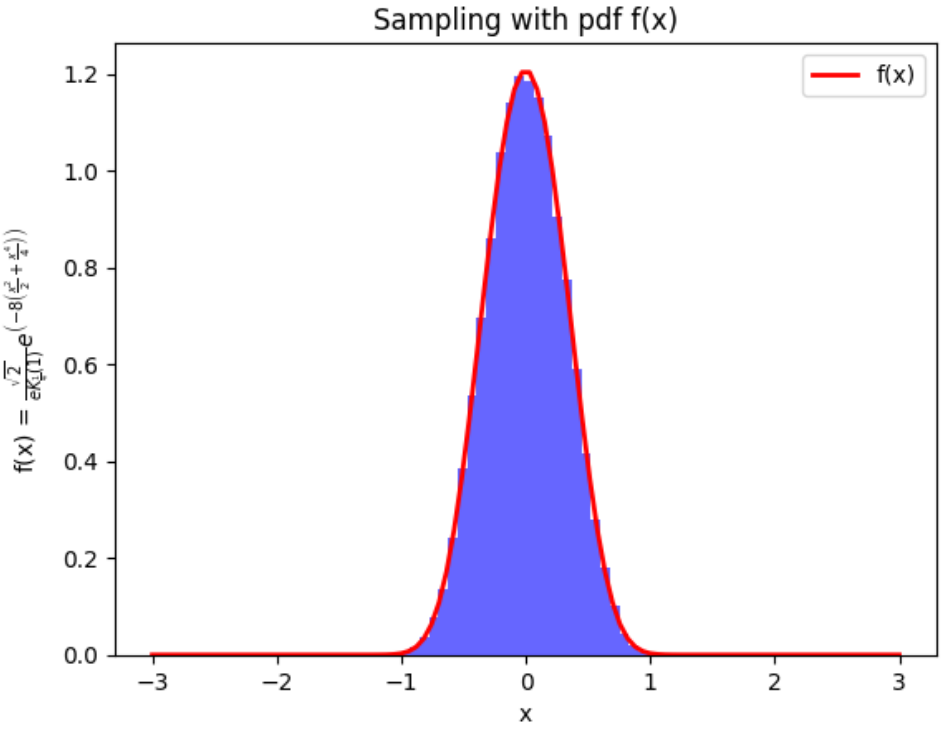

Figure 3: Caption

3 Lecture 3

3.1 Exercise 3.1

(i) The estimation of the integral with Crude Monte Carlo method is done by first sampling random x_i values in $[0, \pi/2]$ uniformely for $i = 1, .., N$, where $N = 50000$. Then the estimate is:

$$I_{MC} = \frac{\pi}{2} \frac{1}{N} \sum_{i=0}^{N-1} \sin(x_i) \tag{4}$$

In the following image, th with its uncertainty calculated as:

$$\sigma_{I_{MC}} = \frac{\pi}{2} \sqrt{\frac{\frac{1}{N} \sum_{i=0}^{N-1} \sin(x_i)^2 - \left(\frac{1}{N} \sum_{i=0}^{N-1} \sin(x_i)\right)^2}{N}} \tag{5}$$

Final estimation of the integral with Crude MC method is:

$$I_{MC} = 1.000 \pm 0.002 \tag{6}$$

In the following image, the left panel shows how the estimate of the integral improves as the sample size N increases, while right panel displays how the accuracy value diminishes with N.

(ii) In order to calculate the integral with importance sampling, a particular probabilty density function $g(x)$ needs to be chosen, and as close as the integran function as possible.

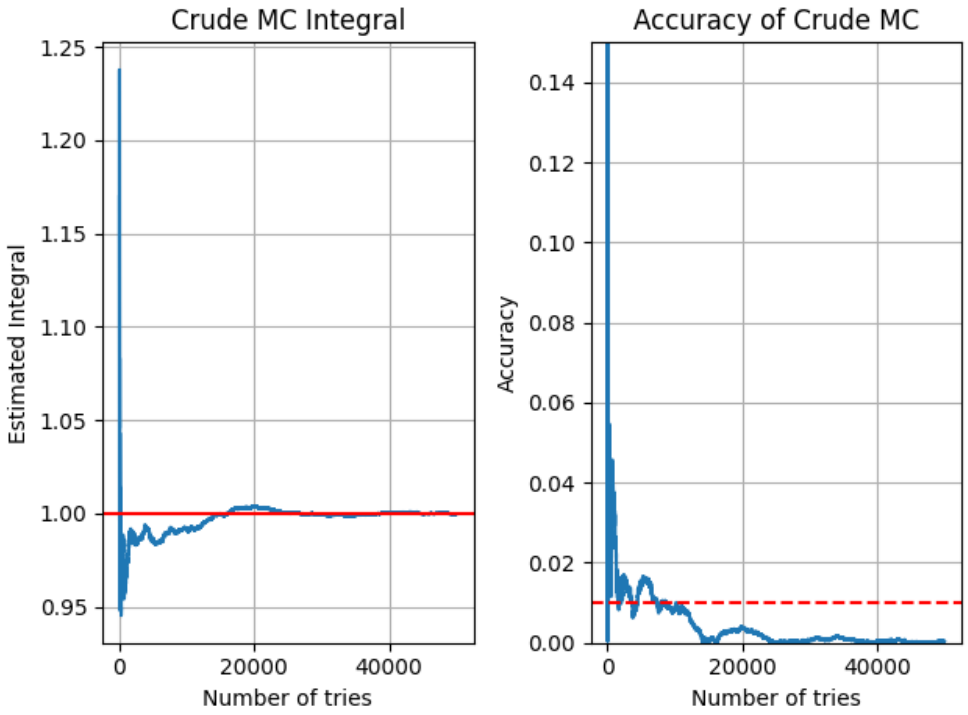

Figure 4: left: estimate of the integral as a function of the size of the sampling. the red line represents the analytical value, which is I = 1. Right: values of the accuracy as a fuction of the sample size again; the dashed red line corresponds to an accuracy of 1%. As one can see, a number around N = 8000 is necessary to reach it

In our case, $g(x) = a + bx^2$. Optimal parameters a and b are chosen to be derived from analytical requirements: first constraint is the normalization, hence: $a\frac{\pi}{2} + b\frac{\pi^3}{24} = 1$. From this, one gets $12a\pi + b\pi^3 = 24$ and then $b = \frac{12(2-a\pi)}{\pi^3}$. At this point $g(x) = a + \frac{12(2-a\pi)\,x^2}{\pi^3}$. The other constraint can come from the imposition that $g(\frac{\pi}{2}) = sin(\frac{\pi}{2} = 1)$ in order to get a distribution which is more similar to the original function. Therefore $1 = a + \frac{12(2-a\pi)}{\pi^3}\,\frac{\pi^2}{4}$. One gets then $a = (\frac{6}{\pi} - 1)\frac{1}{2}$. Final form for the pdf $g(x)$ is

$$g(x) = a + \frac{12(2 - a\pi)\,x^2}{\pi^3} \qquad \text{with } a = \left(\frac{6}{\pi} - 1\right)\frac{1}{2} \tag{7}$$

To sample from $g(x)$, it is possible to apply Von-Neumann rejection method: the chosen probability density function is $h(x) = C\big(a+2\big(\frac{1-a}{\pi}\big)x\big)$ with $C = \frac{4}{\pi(1+a)}$. The cumulative distribution function associated to $h(x)$ is $H(x) = C\big(ax + \big(\frac{1-a}{\pi}\big)x^2\big)$.
Applying the inversion method, $\xi = C\big(ax + \big(\frac{1-a}{\pi}\big)x^2\big)$ it derives $\big(\frac{1-a}{\pi}\big)x^2 + ax - \frac{\xi}{C} = 0$ from which

$$x = \frac{-a + \sqrt{a^2 + \frac{4\xi}{C}\big(\frac{1-a}{\pi}\big)}}{2\big(\frac{1-a}{\pi}\big)} \quad \text{so that } x \sim h(x) \tag{8}$$

where the solution of x with a $+$ sign was chosen since the integration range is positive.

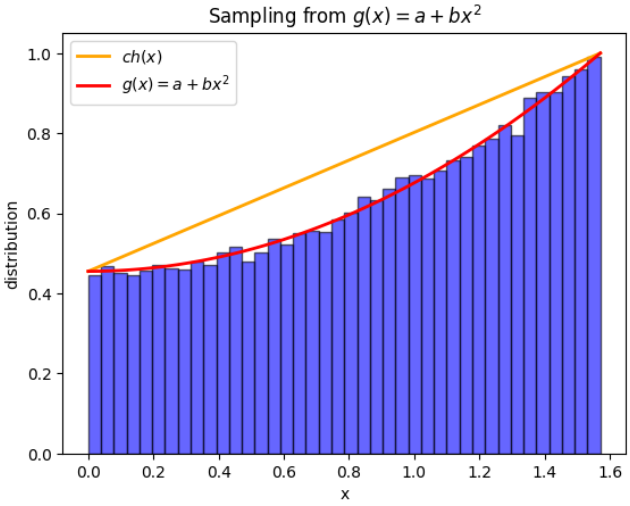

Figure 5: rejection method applied to sample from $g(x) = a + bx^2$ using as additional pdf $h(x)$.

Sampling x as above and a random number $\xi_2 \in [0,1]$, if $\xi_2 \geq \frac{g(x)}{ch(x)}$ with $c = 1/C = \frac{\pi(1+a)}{4} \gtrsim 1$, the extracted x is rejected, otherwise x is stored and then $x \sim g(x)$.
When the sample of N variables $x_i \sim g(x)\,,\ i = 1, ..., N$ is completed, then the integration with importance sampling can be performed: a new variable is defined as $\frac{sin(x_i)}{g(x_i)}$, then

$$I_{IS} = \frac{1}{N}\sum_{i=0}^{N-1}\frac{\sin(x_i)}{g(x_i)} \quad \text{where } x_i \sim g(x) \tag{9}$$

$$\sigma_{I_{IS}} = \sqrt{\frac{\frac{1}{N}\sum_{i=0}^{N-1}\left(\frac{\sin(x_i)}{g(x_i)}\right)^2 - \left(\frac{1}{N}\sum_{i=0}^{N-1}\frac{\sin(x_i)}{g(x_i)}\right)^2}{N}} \tag{10}$$

3.2 Exercise 3.2

(a) To find analytically the value of $\langle f\rangle_\rho$, the integral is computed as following:

$$\begin{aligned}\langle f\rangle_\rho &= \frac{1}{\sqrt{2\pi}}\int_{\mathbb{R}}\big(e^{-\frac{x^2}{2}-\frac{9}{2}+3x} + e^{-\frac{x^2}{2}-\frac{36}{2}+6x}\big)e^{-\frac{x^2}{2}}dx \\ &= \frac{1}{\sqrt{2\pi}}(e^{\frac{-9}{2}}\sqrt{\pi}e^{\frac{9}{4}} + e^{-18}\sqrt{\pi}e^{\frac{36}{4}}) \ \leftarrow \text{Gaussian integrals} \\ &= \frac{1}{\sqrt{2}}(e^{-\frac{9}{4}} + e^{-9}) \approx 0.0745\end{aligned} \tag{11}$$

(b) The integral can be computed numerically too. A first estimation is done with Crude Monte Carlo method: a sample of size $N = 10^3$ of numbers x_i is generated according to a normal distribution $\mathcal{N}(0,1)$; then:

$$\text{Est}_{MC}\langle f\rangle_\rho = \frac{1}{N}\sum_{i=0}^{N-1} f(x_i) \quad x_i \sim \rho(x) = \mathcal{N}(0,1) \tag{12}$$

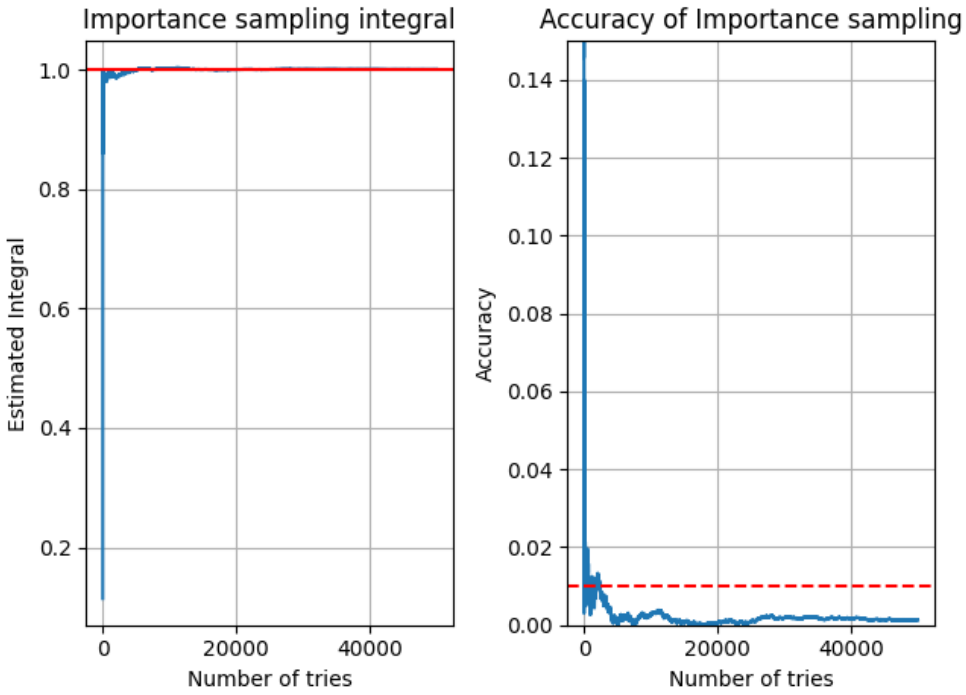

Figure 6: left: estimate of the integral as a function of the size of the sampling. the red line represents the analytical value, which is I = 1. Right: values of the accuracy as a fuction of the sample size again; the dashed red line corresponds to an accuracy of 1%. As one can see, in the case of importance sampling, the desired accuracy is reached before compared to the MC method.

The uncertainty is:

$$\sigma^{MC}_{\langle f\rangle_\rho} = \sqrt{\frac{\frac{1}{N}\sum_{i=0}^{N-1} f(x_i)^2 \; - \; \left(\frac{1}{N}\sum_{i=0}^{N-1} f(x_i)\right)^2}{N}} \tag{13}$$

Since with $N = 10^3$, one can observe that the integral value oscillates visibly, therefore a sample of 200 estimates of the same integral is computed and reported below. The uncertainty is, for each value, 0.005. It can be seen that the sample is quite uniformely distributed around the analytical value, which is represented with a red line.

(c) In this case, importance sampling method is applied. An additional probability distribution function $g(x)$ is chosen, in this case a uniform one, in [-8, -1]. From normalization condition, it derives that $g(x) = 1/7$, a constant. For $N = 10^3$ times, numbers x_i are generated uniformely according to $g(x)$. Then the estimation of the integral becomes:

$$\mathrm{Est}_{IS}\langle f\rangle_\rho = \frac{1}{N}\sum_{i=0}^{N-1}\frac{f(x_i)\rho(x_i)}{g(x_i)} = \frac{1}{N}\sum_{i=0}^{N-1}\frac{f(x_i)\rho(x_i)}{1/7} \qquad x_i \sim g(x) \tag{14}$$

Its error is :

$$\sigma^{IS}_{\langle f\rangle_\rho} = \sqrt{\frac{\frac{1}{N}\sum_{i=0}^{N-1}(f(x_i)\rho(x_i)/g(x_i))^2 \; - \; \left(\frac{1}{N}\sum_{i=0}^{N-1} f(x_i)\rho(x_i)/g(x_i)\right)^2}{N}} \tag{15}$$

As before, since the result oscillation, the calculation is repeated 200 times. The result clearly is incompatible with the expected result. This is due to the choice of $g(x)$; to

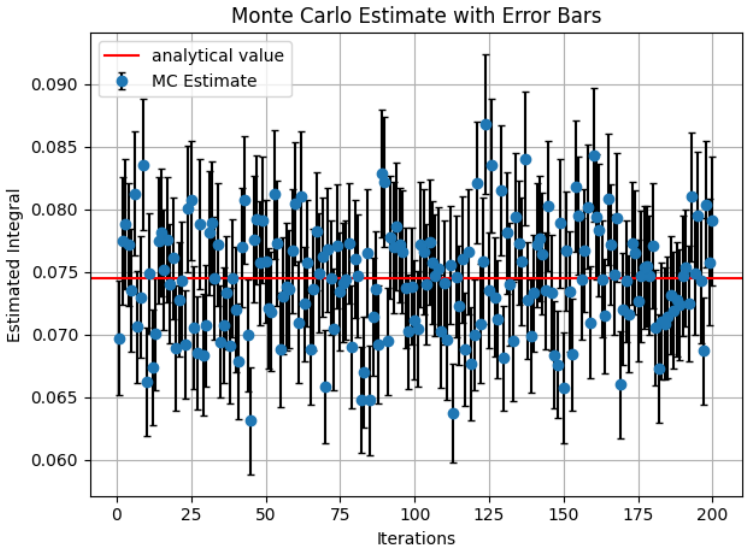

Figure 7: 200 Monte Carlo estimates of $\langle f \rangle_\rho$ with their error bars. Red line represents the analytical value

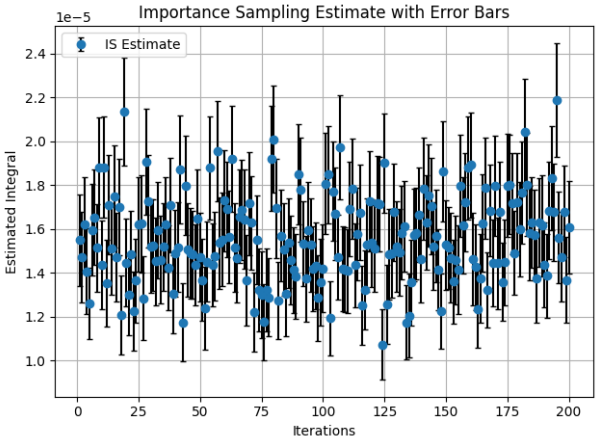

Figure 8

improve the result of the integral, a better choice would be a distribution, say $d(x)$, for which $\rho(x)f(x)/d(x)$ is more constant.

4 Lecture 4: Markov chains

4.1

For both matrices (A) and (B), whose diagraphs can be found in figure 9, the number of possible states is finite, which implies that the states are recurrent. For the first case, all the states communicate with each other having a period of T = 3. For these reasons, matrix (A) describes an irreducible Markov chain. For the second case instead, not all states are communicating, therefore the Markov chain here is not irreducible. For example, from state 2 there is no chance to go to any other state.

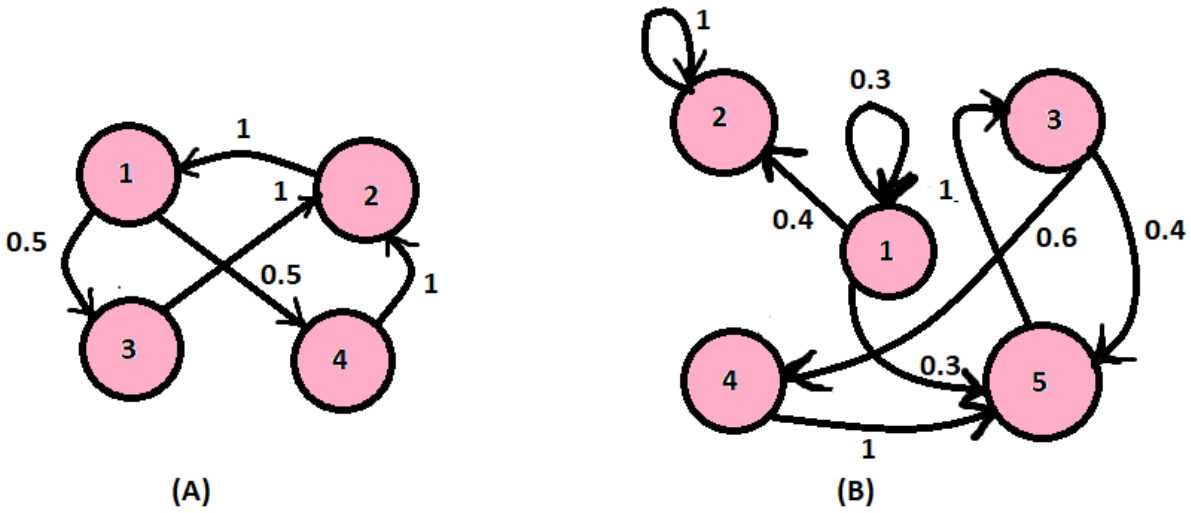

Figure 9

4.2

(a) For this case, looking at the diagraph 10 it is evident that for each state i exists a path connecting it to any other state j, hence this Markov chain is irreducible. Moreover, since $P_{11} > 0$, state 1 is aperiodic and hence all the others (finite space of states).

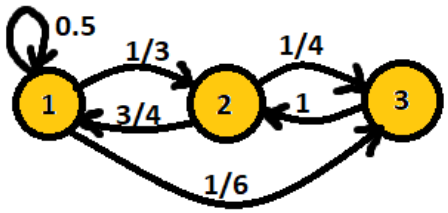

Figure 10

(b) The probability to arrive at state 3 after 2 steps starting from state 1 is $p_{ij}^{(2)}$, so the entry at row i and column j of P^2, so 1/6.
(c) the invariant distribution π indicates the fixed-point of the dynamics described by P. It can be found by the relation $P^T\pi^T = \pi^T$, so it is the eigenvector of eigenvalue 1. An eigenvector is $v = (1/3, 1/3, 1/6)$, so that $P_i^T = (1/2, 1/3, 1/6)$.
Then: $P^\infty = \begin{pmatrix} 1/2 & 1/3 & 1/6 \\ 1/2 & 1/3 & 1/6 \\ 1/2 & 1/3 & 1/6 \end{pmatrix}$

4.3

Starting from the inital state a_0 : A = [w,w] and B = [r, r, r], to go with just an exchange of balls from $a_0 \to a_1$: A = [w, r] and B = [w, r, r] the probability is $p_{01} = 1$; from $a_1 \to a_2 : p_{12} = (1/2)(2/3) = 1/3$; $a_2 \to a_1 = (1/2)(1/3) + (1/2)(2/3) = 1/2$ and so on.

The probability matrix is then: $P = \begin{pmatrix} 0 & 1 & 0 \\ 1/6 & 1/2 & 1/3 \\ 0 & 2/3 & 1/3 \end{pmatrix}$

The probability of a_2 after 3 steps is calculated starting from P^3, getting then $p^{(3)}_{0,2} = 1/6 + 1/9 = 5/18$.

In the limit of many events, again, the invariant distribution is estimated from $P^T\pi^T = \pi^T$. Doing the same calculation as above, searching the eigenvector, the result found is $\pi = (1/3, 1/3, 1/3)$, which then gives $p_{0,3} = 1/3$.

5 Lecture 5: MC simulation of a 2D Ising model by the Metropolis algorithm

In the Ising model, N spins $\sigma = \pm 1$ are located in a lattice with given size L, in unit of spin. The simulation is done with three different size lattices: 25, 50, 75; for each of them the dynamics of the lattice is analyzed at three different temperatures: $T_1 = 0.8\,T_c$, below the critical temperature, $T_2 = 1.05\,T_c$, near the critical tmeperature, and $T_3 = 1.2\,T_c$, above the critical temperature.

In the model, given a configuration $\mathcal{C}$ of spins in the lattice, the hamiltonian is:

$$H(\mathcal{C}) = -J \sum_{\langle r_i r_j \rangle} \sigma_{r_i} \sigma_{r_j} \tag{16}$$

where J is the coupling between the spins and the interaction of a given spin is only limited to the nearest neighbours.

At the beginning, as the initial configuration, a matrix of dimension LxL is created, with entries that are either +1 or -1, randomly picked. The energy of the initial configuration then is calculated as 16.

At this point the Glaubert dynamics is implemented to let the system evolve as a Markov chain: for a number $N = L$xL, a random element of the matrix is picked, namely a position r_k in the lattice, and a spin-flip is attempted as a proposed move. The corresponding difference in energy between the new and the old configuration is $\Delta E = 2\sigma_{r_k} \sum_{n.n(r_k)} \sigma_{r_i}$, where n.n.$(r_k)$ indicates to consider the positions of the nearest neighbours of σ_{r_k}. The Metropolis algorithm then gives the rules to either accept or reject the proposed move. If $\Delta E < 0$ or a if a random number in [0,1] r $< e^{-\Delta E/k_B T}$, the move is accepted, hence the spin is flipped and the overall energy changes by ΔE, otherwise there is no change and the move thus is rejected. To implement periodic boundary conditions: given that `spin = lattice[xi,xj]` provides the spin (-1 or 1) in position $(i.j)$, and that the box of spins has size L, the nearest neighbors of `spin` are given by: `neighbours = (lattice[(xi+1)%L, xj] + lattice[(xi-1)%L, xj] +lattice[xi, (xj+1)%L] + lattice[xi, (xj-1)%L])`.

Equilibration: The equilibration time necessary for the dynamics to converge to the equilibrium state is estimated visually, by observing how the magnetization and the energy evolve with time: when they evidently start to oscillate around an approximately constant value, there the equilibration time is reached. Under the critical temperature, since a ferromagnetic phase is expected, the magnetization per spin should eventually fluctuate around ± 1. Above the critical temperature, in a paramagnetic phase, the magnetization should be close to zero. Near the critical temperature, a more mixed behaviour

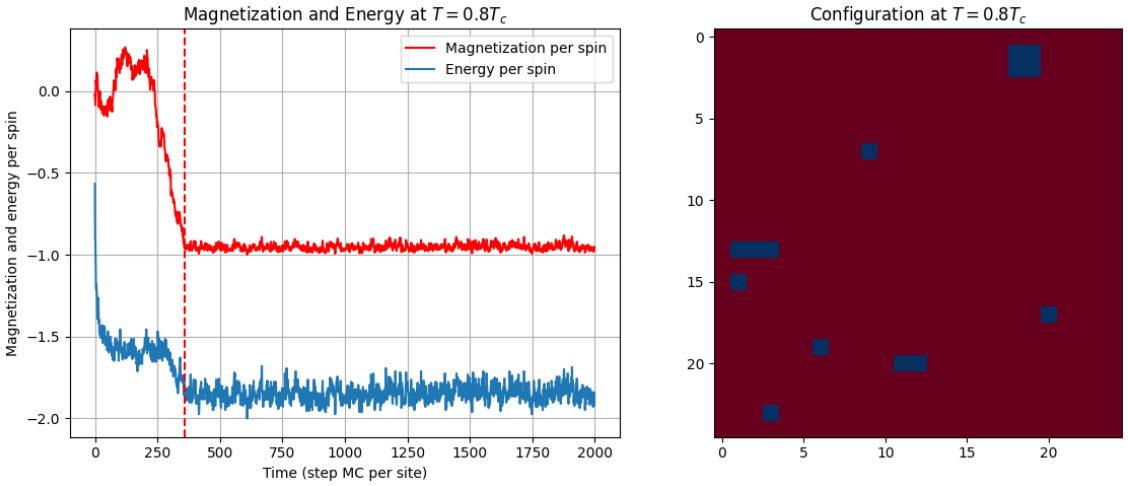

Figure 11: Simulation with L = 25, below the critical temperature.

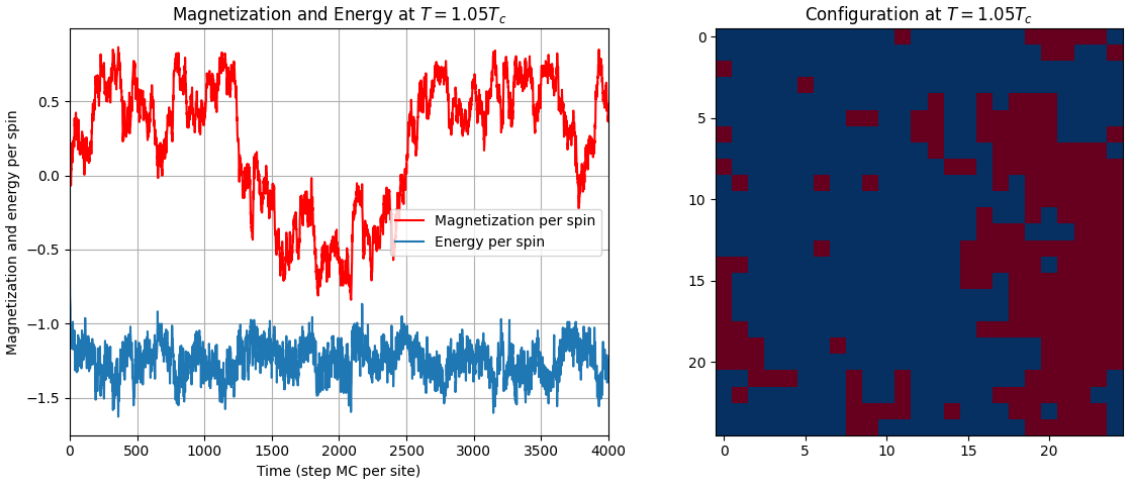

Figure 12: L = 25 and slightly above the critical temperature. Large oscillations are visible in the magnetization.

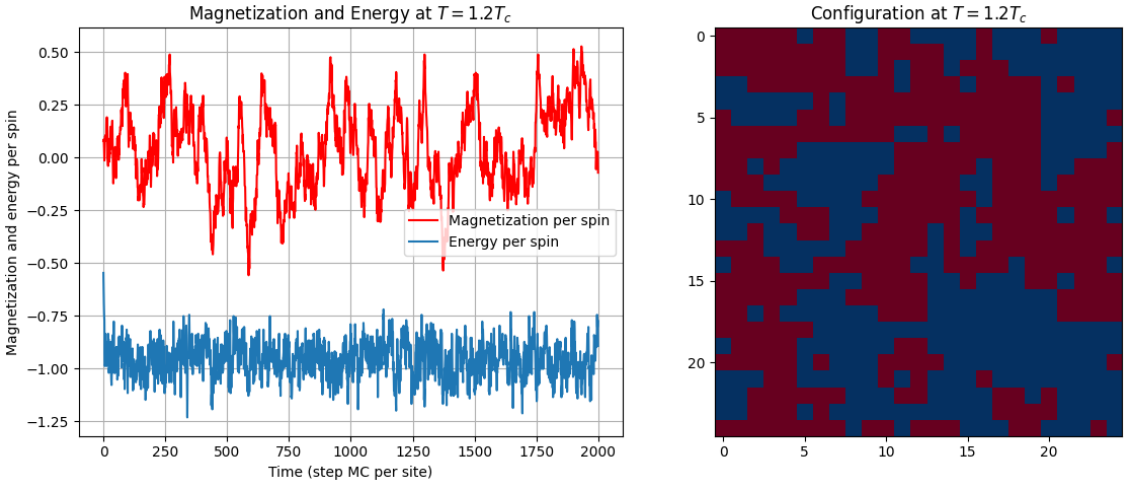

Figure 13: L = 25 and temperature above the critical temperature. Diamagnetic phase visible

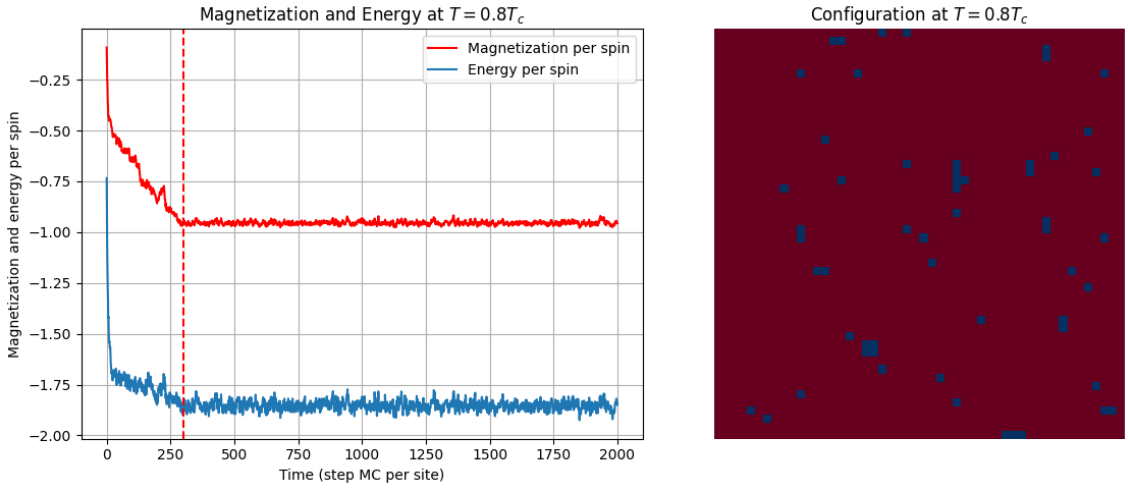

Figure 14: Simulation with L = 50 and below the critical temperature. Here the equilibration is pointed out with a dashed line. Right panel: the color in the lattice represent the sign of the spin. Almost all spins are aligned and have negative sign.

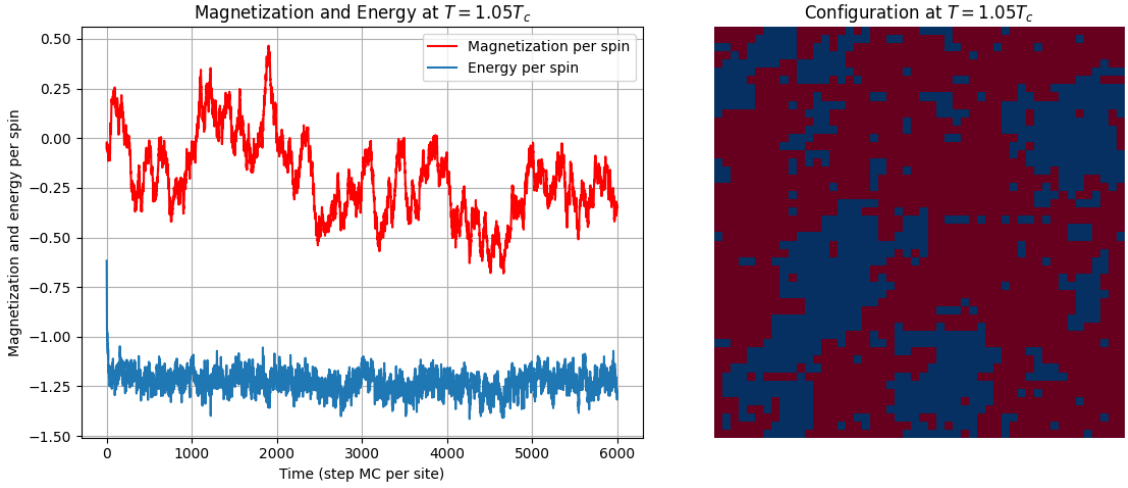

Figure 15: L=50 and T close to the critical temperature. The diamagnetic phase is more visible, but high oscillation in magnetization are present

is observed.

Once the equilibration time τ_{eq} is determined for a given temperature and dimension of the lattice, ensemble averages of the energy per spin, magnetization per spin, specific heat and magnetic susceptibility are computed as:

$$\overline{O}_{t_{max}} = \frac{1}{t_{max} - \tau_{eq}} \sum_{\tau_{eq}}^{t_{max}} O_t \tag{17}$$

In the following table the results are reported. Uncertainties are already corrected with the estimation of the correlation time τ_{corr}.
For the uncertainty of the fluctuations, $N\chi_T$ and C_V, a block average of magnetization and energy is done, dividing the sample in k blocks, so to have also a sample of k variances. The error is estimated then as the standard deviation of the k variances over $\sqrt{N_i}$, with N_i the numerosity of the given i-th block. The division in blocks is chosen according to

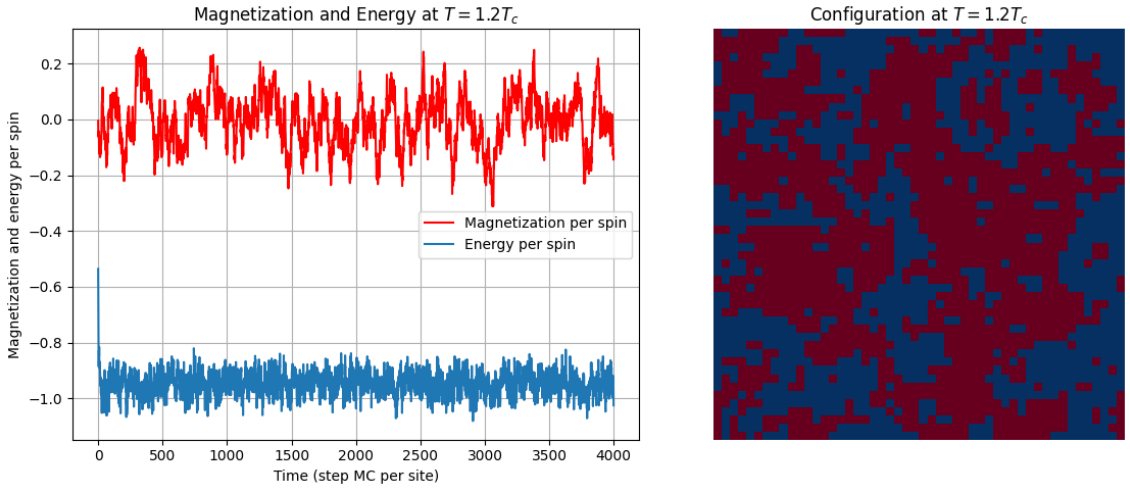

Figure 16: L=50 and T higher than the critical temperature. Diamagnetic phase.

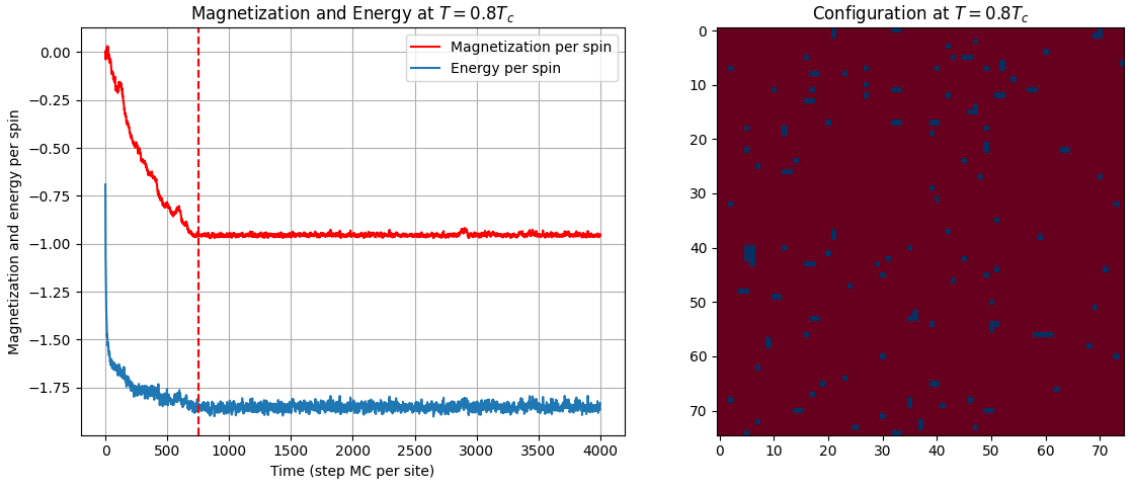

Figure 17: Simulation with L = 75 and below the critical temperature. Here the equilibration is pointed out with a dashed line. Right panel: the color in the lattice represent the sign of the spin. Almost all spins are aligned and have negative sign.

the correlation time: N_i $i = 1, ..., k$ has to be higher than the τ_{corr} to ensure uncorrelated blocks of data.

	$\langle m \rangle \pm \sigma_{\langle m \rangle}$	$\langle E \rangle \pm \sigma_{\langle E \rangle}$	N χ_T	C_V
$T = 0.8\,T_c$ (L = 25)	-0.9526 ± 0.0014	-1.847 ±0.003	0.104 ± 0.006	$(6.7 \pm 0.3) \cdot 10^{-4}$
$T = 1.05\,T_c$ (L = 25)	0.23 ± 0.06	-1.248 ± 0.004	34 ± 11	0.00230 ± 0.00013
$T = 1.2\,T_c$ (L = 25)	-0.04 ± 0.03	-0.958 ± 0.005	6.6 ± 0.7	$(9.3 \pm 0.5) \cdot 10^{-4}$
$T = 0.8\,T_c$ (L = 50)	-0.9548 ± 0.0006	-1.8539 ± 0.0013	0.103 ± 0.008	$(15.4 \pm 0.8) \cdot 10^{-5}$
$T = 1.05\,T_c$ (L = 50)	-0.208 ± 0.09	-1.23098 ± 0.09	18 ± 3	$(4.1 \pm 0.2) \cdot 10^{-4}$
$T = 1.2\,T_c$ (L = 50)	-0.005 ± 0.012	-0.9486 ± 0.0017	(6.32 ± 0.6)	$(2.29 \pm 0.17) \cdot 10^{-4}$
$T = 0.8\,T_c$ (L = 75)	-0.9536 ± 0.0004	-1.8514 ± 0.0007	0.122 ± 0.010	$(7.4 \pm 0.3) \cdot 10^{-5}$
$T = 1.05\,T_c$ (L = 75)	-0.12 ± 0.06	-1.230 ± 0.004	38 ± 10	$(2.6 \pm 0.4) \cdot 10^{-4}$
$T = 1.2\,T_c$ (L = 75)	0.001 ± 0.008	-0.9524 ± 0.0015	7.8 ± 0.9	$(11.1 \pm 0.9) \cdot 10^{-5}$

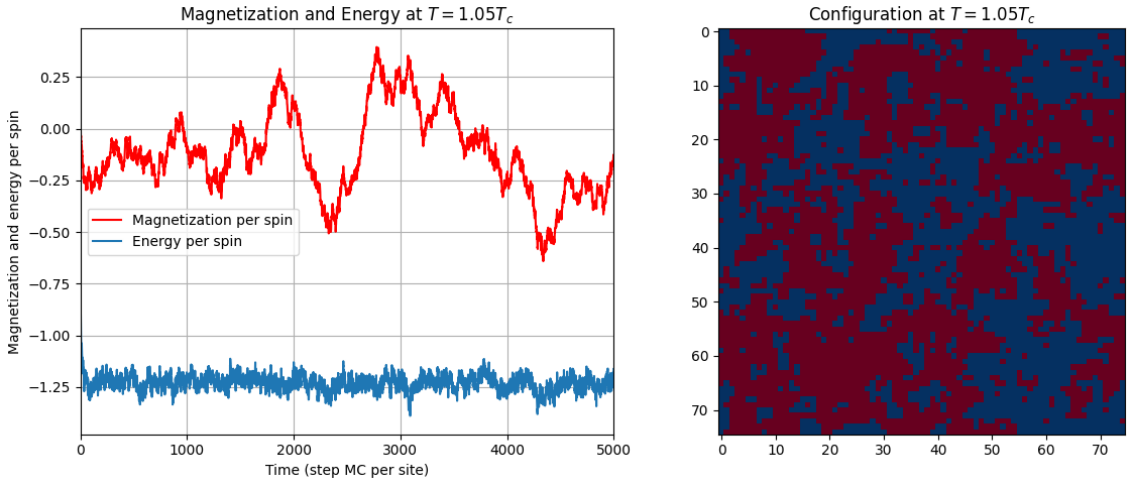

Figure 18: L=75 and T close to the critical temperature. The diamagnetic phase is more visible, but high oscillation in magnetization are present

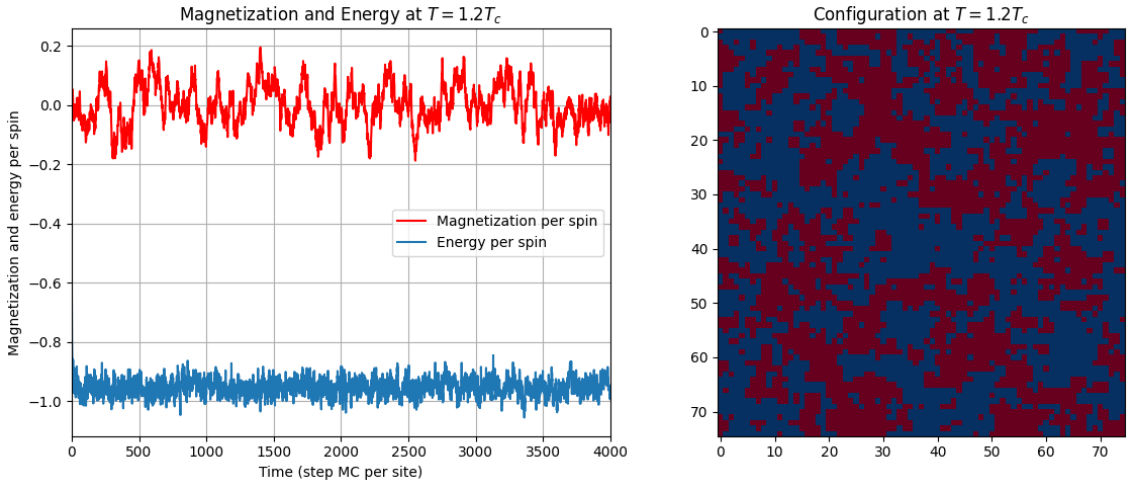

Figure 19: L=75 and T higher than the critical temperature. Diamagnetic phase.

Correlation time: The following step is to estimate the correlation time of the energy per spin and of the magnetization per spin. In order to do so, the correlation functions of those observables, after the equilibration time, are calculated first:

$$C_O(t) = \frac{1}{t_{max} - t} \sum_{t'=0}^{t_{max}-t} O(t')O(t'+t) - \frac{1}{(t_{max}-t)^2} \sum_{t'=0}^{t_{max}-t} O(t') \sum_{t'=0}^{t_{max}-t} O(t'+t) \qquad (18)$$

where $t = 0, 1, ..., t_{sample}$, and t_{sample} is chosen to be a either 15% or 40% of the time of the simulation t_{max}. The latter percentage is chosen for the near-the-critical-temperature cases, for which the autocorrelation time grows fast, and correlation functions take more time to arrive to zero. Correlation functions are then normalized to the first value $C(0)$ and τ_{corr} is estimated with two methods: 1) exponential fit $e^{-t/\tau_{corr}}$ in the region in which correlation is positive and decreasing; 2)

$$\tau_{corr} = \sum_{t=0}^{t_{zero}} \frac{C(t)}{C(0)} \Delta t \quad \text{where } \Delta t = 1 \qquad (19)$$

and t_{zero} is chosen according to the graph: it is the time at which the correlation function arrives at zero. This choice is due to the fact that correlation oscillates greatly around 0 with some irregularities, being the statistics quite limited.
The final estimation of $\tau_{corr,M}$ and $\tau_{corr,E}$ are sometimes chosen as an average of the results from the two methods when those are very similar; other times, when the fit method is more accurate, the final τ_{corr} is chosen from the result of that procedure; conversely, when the fit curve is visibly not well compatible with data, second method estimation is preferred.
With the final correlation times for magnetization and energy, new error bars for the averages of those observables are computed as follows:

$$\sigma_{\overline{O},\,final} = \sqrt{\frac{\frac{1}{N}\sum_{\tau_{eq}}^{t_{max}} O_t^2 \;-\; \left(\frac{1}{N}\sum_{\tau_{eq}}^{t_{max}} O_t\right)^2}{N} \cdot \left(1 + \frac{2\tau_{corr,O}}{\Delta t}\right)} \tag{20}$$

6 Lecture 6: Multiple Markov chains

The simulated grafted polymer here has 15 monomers, with the first one being anchored in position (0,0). The system evolves in a 2d plane. Initial configuration for the polymer is chosen to be the vertical one, where $x_i = 0$ for $i = 0, ..., N-1 = 0, ..., 14$ and $y_{i+1} - y_i$ being randomly picked in the allowed bond length range [c, 1.3c] = $[l_{min}, l_{max}]$, where c = 1. The given configuration is copied for all temperatures. In fact, the polymer itself at a given temperature is an array with N entries, one for each monomer, each entry being a 2d vector in which the monomer coordinates are stored and updated at each MC step.

```
    # initial configuration of the polymer
monomers = [(0, 0)]
for i in range(1, N):  # initial vertical configuration
    distance = random.uniform(l_min, l_max)
    x_new = 0
    y_new = monomers[i - 1][1] + distance
    monomers.append((x_new, y_new))

[...]

for i in range(len(temperatures)): # Creating an array of initial configurations,
one for eache T
        initial = monomers.copy()
        polymer.append(initial)
```

From this initial configuration, the evolution of the system proceeds via Monte Carlo simulation using the Metropolis algorithm. Possible moves are 1) N attempted small shifts of N randomly picked monomers and 2) rigid rotations of the monomers around a randomly given monomer (pivot). 1 MC step is given by N tried shifts followed by 1 attempted rotation.
For the 1^{st} type of block of moves, the shift, displacements dx, dy are generated according to a normal distribution with 0 mean and 0.15c variance. The choice of these particular parameters leads to acceptance rates between $\sim$ 34% and $\sim$ 40% while sampling at

different temperatures. As an example, while choosing a variance of 0.3, the resulting acceptance rates drop to $\sim 14\%$ - $\sim 18\%$. Before the move is subjected to the Metropolis test, it does have to pass the geometric conditions: consecutive bonds must range between $[l_{min}, l_{max}]$ and the distance between a given monomer and each of the other monomers must be greater than l_{min}; additionally, $y_i > 0$ for $i = 1, ..., N-1$. Here I report the control functions I used in the code:

```
    # distances among all monomers + y > 0 constraint
def valid(index, new_mon, polymer):
    for i in range(len(polymer)):
        if new_mon[1] < 0:
            return False
        if i != index:
            distance = ((new_mon[0] - polymer[i][0])**2 + (new_mon[1] - polymer[i][1])
            if distance < c:
                return False
    return True


# distance for consecutive monomers, after the small shift
def valid_small(i, new_mon, polymer):
    d1 = ((new_mon[0] - polymer[i - 1][0])**2 +
    (new_mon[1] - polymer[i - 1][1])**2)**0.5
    if i + 1 < len(polymer):
        d2 = ((new_mon[0] - polymer[i + 1][0])**2 + (new_mon[1] - polymer[i + 1][1])*
        if d1 < l_min or d1 > l_max or d2 < l_min or d2 > l_max:
            return False
    else:
        if d1 < l_min or d1 > l_max:
            return False
    return True
```

If the move passes these geometrical constraints, Metropolis test is done. The energy of the polymer in this model comes from the spatial configuration of the system: if $0 < y_i < 0.5 \rightarrow E = E - \epsilon$ with $\epsilon = 1$. The following function executes the block of shift moves on the polymer.

```
def block_shift(polymer, T):
    for _ in range(N):
        i = random.randint(1, N - 1)
        dx, dy = np.random.normal(0, 0.15, 2)
        new_mon = (polymer[i][0] + dx, polymer[i][1] + dy)

        if valid(i, new_mon, polymer) and valid_small(i, new_mon, polymer):
            old_mon = polymer[i]

            if test_Metropolis_shift(old_mon, new_mon, epsilon, T):
                polymer[i] = new_mon
            else:
```

```
                polymer[i] = old_mon
    return polymer
```

For the rotation move now, a monomer is picked randomly between $i = 0$ and $i = N - 2$ (last monomer has index $N - 1$). The monomers located after it will be rotated (provided that the move is accepted) rigidly by an angle which is generated with a uniform distribution in a symmetric interval $[-\theta, \theta]$ to also ensure the detailed balance condition; the choice of θ is done according to the temperature: for temperatures T smaller than 0.45 $\theta = \pi/4$, for $0.45 < T < 0.65$, $\theta = \pi/2$ and finally or $T > 0.65$, then $\theta = \pi$; this is motivated by the fact that, for different temperatures, the acceptance rate changes as the chosen angle changes. In particular, for smaller temperatures, large angles lead to very small acceptance rates ¡ 10%, while for higher temperatures choosing small angles results in ¿ 50% rates of acceptance. See figure 20 in which the rates are plotted. To

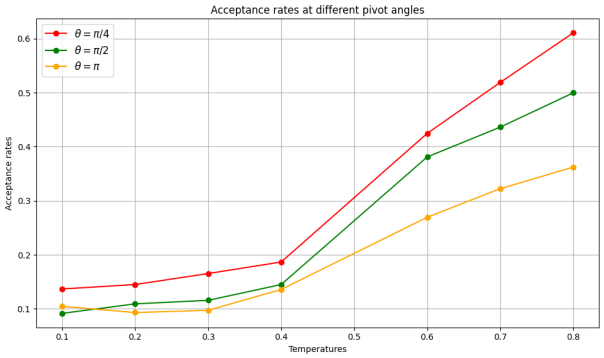

Figure 20: acceptance rates at different temperatures for different pivot angle. In the simulation, for lower temperatures (< 0.45), $\pi/4$ is chosen; for temperatures in the middle, $\pi/2$ is used; and finally, for higher temperatures (> 0.65), the chosen angle is π

find the rotated coordinates, a rotation matrix is applied (see the function in the code reported below). A new "branch" of rotated monomer is built and attached to the rest of the polymer that does not rotate (i=0,...,j, where j is the index of the pivot-monomer). Geometrical check of the now new candidate polymer are done and if they pass, Metropolis test is done on the rotated branch. Below the function regarding the rotation move is reported.

```
def block_pivot(polymer, T):
# list to store the new positions of the monomers belonging to a rotated branch
    new_branch = []

# random postion of the pivot excluding the last monomer
    j = random.randint(0, N - 2)

    if T < 0.45:
        angle = random.uniform(-np.pi/4, np.pi/4)
    elif T >= 0.45 and T < 0.65:
        angle = random.uniform(-np.pi/2, np.pi/2)
    else:
        angle = random.uniform(-np.pi, np.pi)
    pivot = polymer[j]
```

```
 # first element of the new branch after rotation is initialized to the pivot itself
    new_branch.append(pivot)
    E_old, E_new = 0, 0

# consider each monomer located after the pivot
    for k in range(j + 1, N):
    # calculation of the enrgy before the rotation:
        if polymer[k][1] < h:
            E_old -= 1
    # new coordinates after rotation:
        x_rel, y_rel = polymer[k][0] - pivot[0], polymer[k][1] - pivot[1]
        x_rot = x_rel * np.cos(angle) - y_rel * np.sin(angle)
        y_rot = x_rel * np.sin(angle) + y_rel * np.cos(angle)
    # candidate for the new, rotated monomer:
        rot_j = (pivot[0] + x_rot, pivot[1] + y_rot)
        new_branch.append(rot_j)

    polymer1 = polymer[:j] + new_branch
    #Verifica la validità e aggiorna E_new se rot_j è accettabile
    for k in range(j + 1, N):
        if valid(k, polymer1[k], polymer1) and valid_small_1(k, polymer1):
            if polymer1[k][1] < h:
                E_new -= 1
        else:
            return polymer

    # Metropolis
    if E_new < E_old or random.random() < np.exp(-(E_new - E_old) / T):
        return polymer1
    return polymer
```

As the system evolves in time, for temperatures small enough, a phase transition is visible: the polymer is absorbed by the wall at $y = 0$. An example of final configurations of the system at different temperatures is reported in figure 21.

Now, the Multiple Markov Chain method is applied as follows:

```
for  t in range(1, time_steps + 1):
    for j in range(len(temperatures)):
    # For each temperature at a given time t, do the block of moves
        T_index = temperature_indices[j]
        current_T = temperatures[j]
        # BLOCK OF MOVES
        polymer[T_index] = block_shift(polymer[T_index], current_T)
        polymer[T_index] = block_pivot(polymer[T_index], current_T)

        [...]

        # ----------------------  [  SWAP  ]  ----------------------------
```

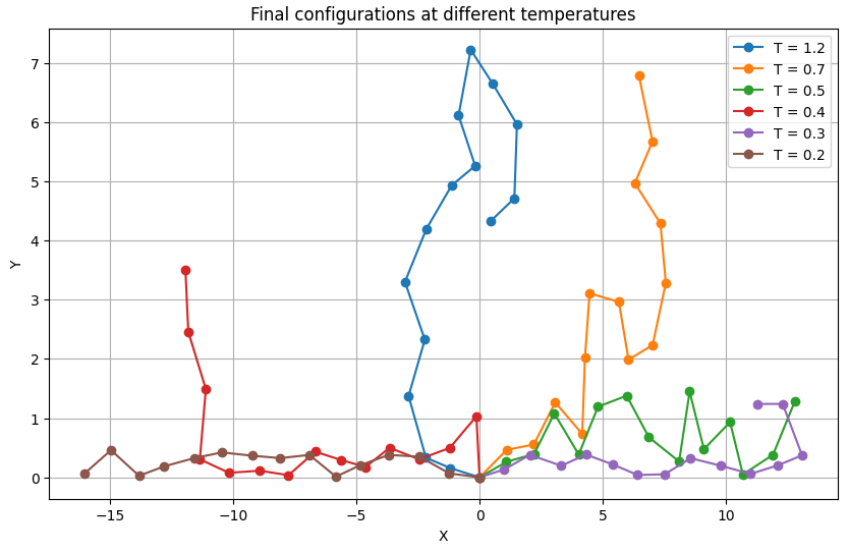

Figure 21: Final configurations of the polymers at different temperatures

```
# After the block of moves is done for each temperature and for some times,
I try to swap
    if t % len(temperatures) == 0:
        k = random.randint(0, len(temperatures) - 2)
        # I assign to i the index of the extracted temperature
        i = temperature_indices[k]
        # i_next has the ndex of the temperature following the one extracted
        i_next = temperature_indices[k + 1]
        T1, T2 = temperatures[k], temperatures[k + 1]
        E1, E2 = total_energy(polymer[temperature_indices[k]]),

        total_energy(polymer[temperature_indices[k + 1]])

    delta = (1 / T2 - 1 / T1) * (E2 - E1)  # assuming T2 < T1
    swap_attempts += 1
    swap_tries[k] += 1
    if E2 > E1 or random.random() < np.exp(delta):
        temperature_indices[k] = i_next
        temperature_indices[k+1] = i
        successful_swaps += 1
        swap_ok[k] += 1
    [...]
```

A list of temperature indices is created. Index k is generated, so T_k and T_{k+1} are the candidate temperatures. The idea, if the swap happens, is to evaluate a configuration C_k generated at temperature T_k as if it "belonged" to T_{k+1}: the block of moves is then executed, the following time step, on C_k but the Metropolis test is done using T_{k+1}. Energy of C_k is evaluated and then stored as $E_{T_{k+1}}$. The same is done with the other observables. With respect to the probability of swapping, this is regulated by a Metropolis algorithm (see function above). The swapping rate for each couple of temperatures must be higher than 20 %. Temperatures then are chosen to realize that condition: they

cannot be too much far apart, otherwise the probability of swapping drops. To ensure that the swap happens with the expected frequency, also the energy histogram is plotted reported in figure 22. As one can see, there is an amount of overlapping among different temperatures, especially between two neighbouring ones. If there was no overlap, no swapping could be possible.

To further study the evolution of the system, the average energy, the average height of the

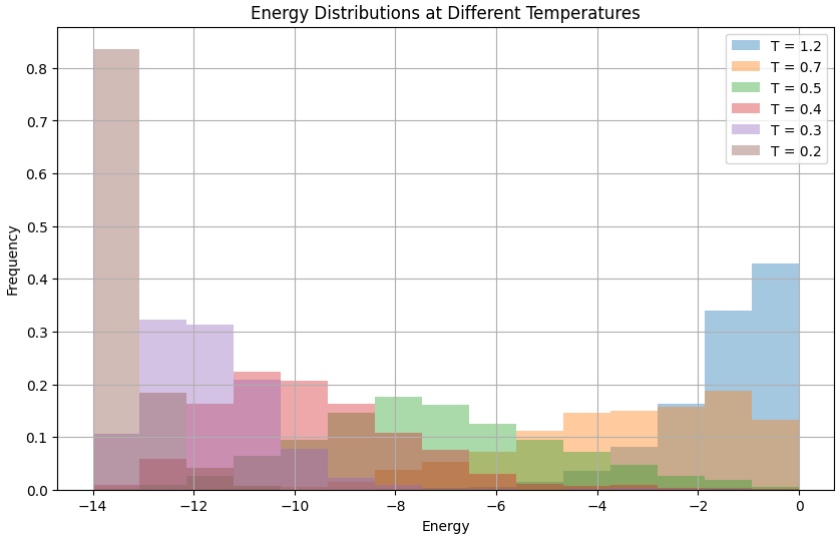

Figure 22: Histogram of the energies at different temperatures. Overlapping between distributions is clearly visible.

last monomer, and the average end-to-end distance for each temperature are calculated and plotted. Figure 23 shows the results obtained. Since below the critical temperature the polymer tends to be adsorbed, it follows that the energy calculated as described above will actually tend to be very negative, as can be seen in the graph; moreover, it is expected, as then actually happens, that the average last monomer height will tend to cancel out, while the end-to-end distance will tend to maximize. In fact, for high temperatures, the polymer moves freely in space, even twisting. Averages are calculated discarding the first $15 \cdot 10^3$ data. The complete data (averaged every 100 steps) are reported in figure 24.

In figure 23 it can be observed that there is a region of transition between the two phases of the polymer, located in between $T = 0.4$ and $T = 0.6$. In this range, the critical temperature is expected to be found. This can be seen better in the plot of the fluctuations of those observables $\langle O^2 \rangle - \langle O \rangle^2$. Looking at figure **??**, it can be suggested that the maximum of the fluctuations lies in between $T = 0.5$ and $T = 0.7$.
Another parameter that can be looked at when trying to identify the temperature region into which the critical temperature falls is the autocorrelation time, because it is supposed to grow while approaching the critical temperature. Correlation functions are calculated for the observables of interest after equilibration as an average of 5 correlation functions calculated in time ranges of 500 time steps.

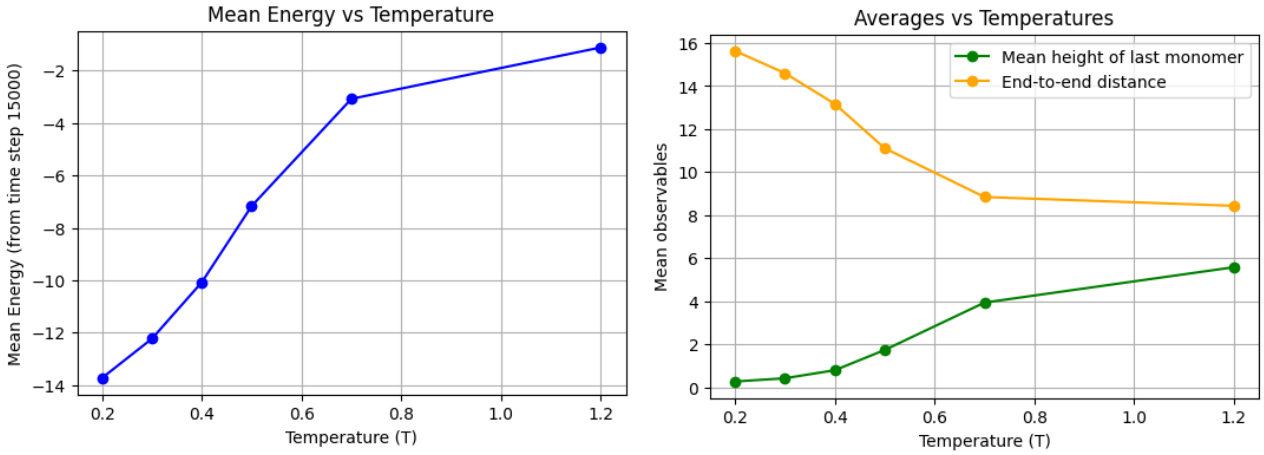

Figure 23: Left: mean energy per temperature. Right: mean end-to-end distance and mean height of the last monomer

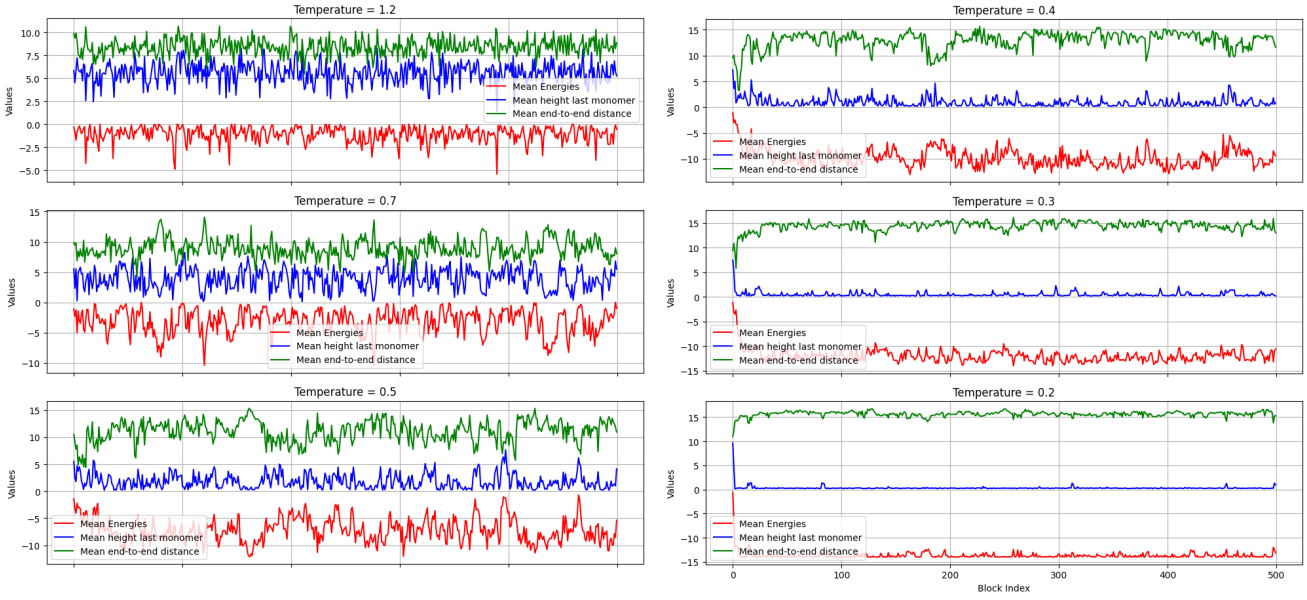

Figure 24: Observable of interest, block averages every 100 steps. Simulation run of $5 \cdot 10^4$ steps

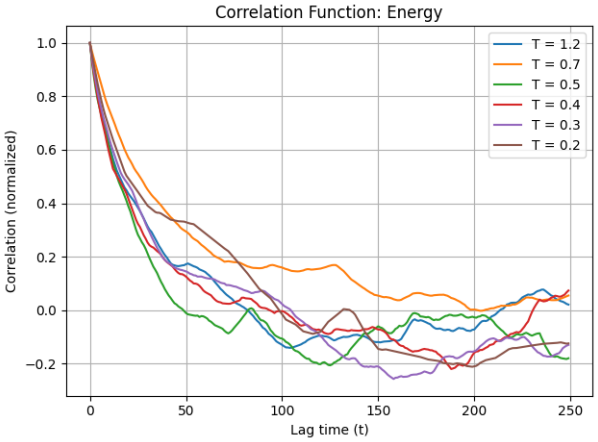

Figure 25: Correlation function of the energy

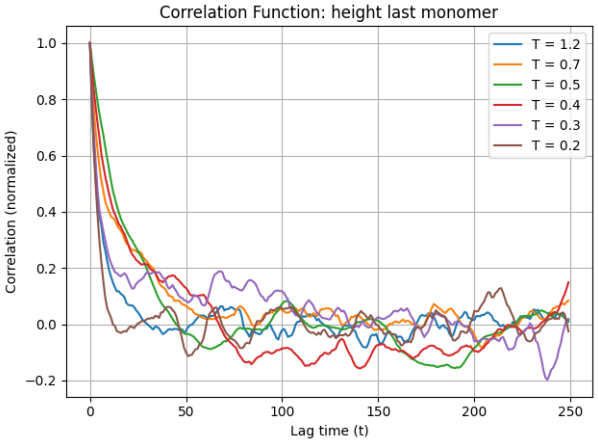

Figure 26: Correlation function for the height of the last monomer

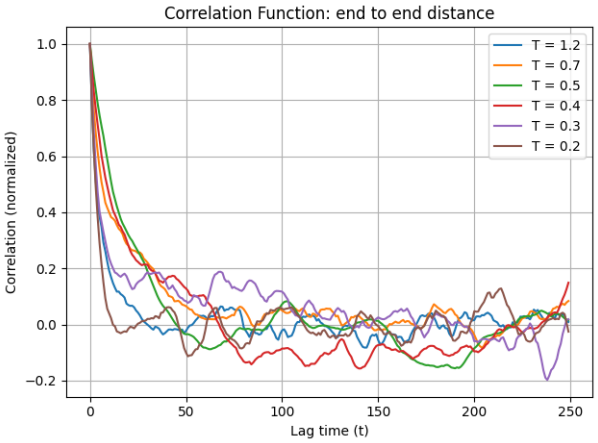

Figure 27: Correlation function for the End-to-end distance

7 Lecture 7: Gillespie algorithm

The Brusselator is a model for a simple chemical system that exhibits oscillatory behavior. It is defined by the following transition rules for the state (X, Y)

$$\begin{cases} w_1 = a\Omega \quad for\,(X,Y) \to (X+1,Y) \\ w_2 = X \quad for(X,Y) \to (X-1,Y) \\ w_3 = \frac{1}{\Omega^2} X(X-1)Y \quad for(X,Y) \to (X+1,Y-1) \\ w_4 = bX \quad for\,(X,Y) \to (X-1,Y+1) \end{cases} \tag{21}$$

The escape rate λ_C is given by the sum of all transition rates. According to Gillespie algorithm, a random waiting time τ generated according to $p_C(t) = \lambda_C e^{-\lambda_C t}$ (from the inverse of the cumulative function of p_C). Then, starting from a configuration $C = (X, Y)$, the system jumps from C to another state C' that has a probability $p(C'|C) = w_{CC'}/\lambda_C$. The sttate C' is chosen extracting a random number $\xi \in [0, 1]$ and using the cumulative distribution of $p(C'|C)$. These two steps of the algorithm are repeated until $\sum_i \tau_i$ exceeds a maximum time t, in this case $t = 30$.
The system is simulated with different volumes Ω. We can see that the oscillations progressively become less significant while increasing the volume, so the system behaves more "deterministically". The fluctuations in small systems are significant because each reaction causes a relatively large change in the state, while if the volume increases, those changes become less and less important.
In figure 28 29 the stochastic limit cycle and the corresponding oscillations are plotted, for volumes $\Omega = 10^2, 10^3, 10^4$.

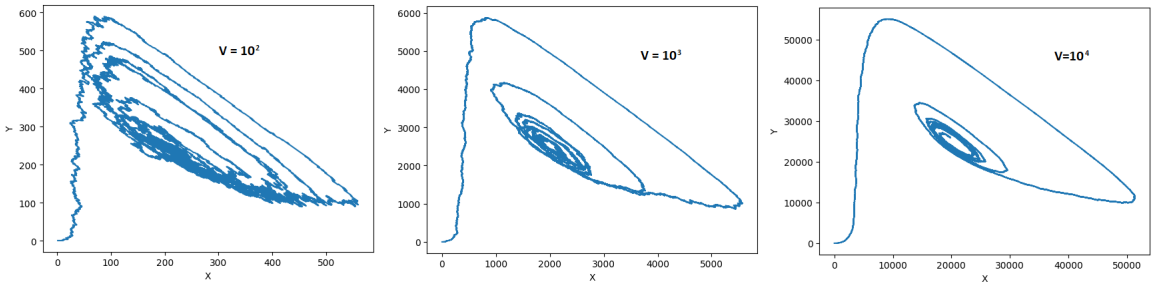

Figure 28: Stochastic limit cycle for different volumes

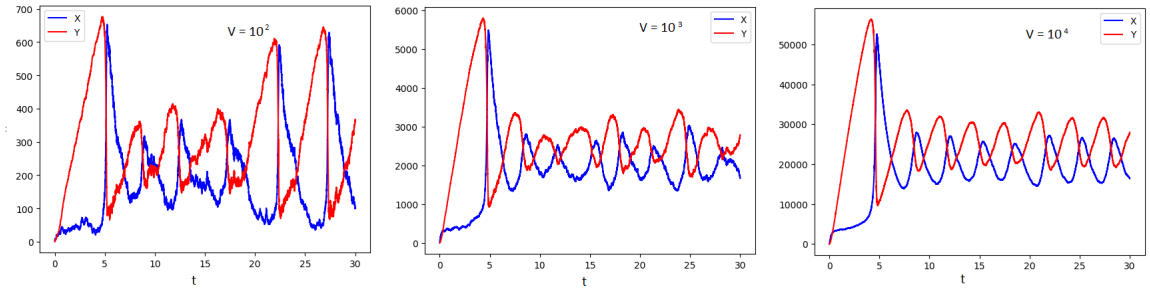

Figure 29: Oscillation in population during simulation time

8 Lecture 8: Off-lattice Monte Carlo

Reduced units (pen and paper)
Parameters: $\sigma_{Ar} = 3.41$Å and $\sigma_{Kr} = 3.38$Å; characteristic energy scale $\frac{\epsilon_{Ar}}{k_B} = 119.8$ K; $\frac{\epsilon_{Kr}}{k_B} = 164.0$ K. Since the reduced temperature is related with the temperature in SI units as $T^* = \frac{k_B T}{\epsilon}$, then:

$$T_{Ar} = T^* \frac{\epsilon_{Ar}}{k_B} = 239.6K \tag{22}$$

$$T_{Kr} = T^* \frac{\epsilon_{Kr}}{k_B} = 328K \tag{23}$$

Now, having a time step of $\Delta t = 0.001\tau$, with τ a typical time scale that can be expressed as $\tau = \sqrt{\frac{m\sigma^2}{\epsilon}}$, with σ being a characteristic size of the particle. For the present exercise: $\sigma_{Ar} = 3.41$ Å $= 3.41 \cdot 10^{-10} m$, $\sigma_{Kr} = 3.38$ Å $= 3.38 \cdot 10^{-10} m$, $m_{Ar} = 66.314 \cdot 10^{-27}$ kg, $m_{Kr} = 139.108 \cdot 10^{-27}$kg; $\epsilon_{Kr} = 164\, k_B = 2.26 \cdot 10^{-21} J$, $\epsilon_{Ar} = 119.8 k_B = 1.65 \cdot 10^{-21} J$. From these paremeters it follows that:

$$\tau_{Ar} = \sqrt{\frac{m_{Ar}\sigma_{Ar}^2}{\epsilon_{Ar}}} = 2.16 \cdot 10^{-12} s \tag{24}$$

$$\tau_{Kr} = \sqrt{\frac{m_{Kr}\sigma_{Kr}^2}{\epsilon_{Kr}}} = 2.65 \cdot 10^{-12} s \tag{25}$$

Therefore: $\Delta t_{Ar} = 0.001\tau_{Ar} = 2.16 \cdot 10^{-15} s$ and $\Delta t_{Kr} = 0.001\tau_{Kr} = 2.65 \cdot 10^{-15} s$

Off lattice Monte Carlo of Lennard Jones particles
This simulation is carried out with 100 particles in a cubic box With periodic boundary conditions. Initially, the particles are placed randomly in in the box. Distances are computed and stored in a matrix D 100x100 whose entries $D_{ij} = D_{ji}$ give the distance between particle i and j. Of course $D_{ii} = 0$ for every i. Then a given MC steps consists of a global move of N = 100 attempted displacements of the randomly selected particle, accepted or rejected according to Metropolis algorithm: the energy at this stage is only given by the LJ potential. Displacements are uniformely generated in an interval $[-\Delta, \Delta]$. Since the simulation is done for different densities, the choice of Δ depends on them and it is based on keeping the rate of acceptance of the move between 25 % and 45 %.

ρ at T = 0.9	Δ	ρ at T = 2	Δ
$\rho \geq 0.75$	0.15	$\rho \geq 0.75$	0.15
$0.3 \leq \rho < 0.75$	0.25	$0.35 \leq \rho < 0.75$	0.35
$0.2 < \rho < 0.3$	0.45	$0.2 < \rho < 0.35$	1
$\rho \leq 0.2$	0.6	$\rho \leq 0.2$	1.5

Table 1: maximum displacement move according to density of LJ fluid

After 2000 initial steps of equilibration, another run of 1000 steps is done: this time, for each step the term $Vir = \sum_{i=1}^{N} \sum_{j=i+1}^{N} \vec{r_{ij}} \cdot \vec{F_i} = \sum_{i=1}^{N} \sum_{j=i+1}^{N} (-dV/dr)|\vec{r_{ij}}|$ is calculated. After that, an average is calculated and then pressure is retrieved from the equation of state as:

$$P_{eq} = \rho k_B T - \frac{1}{3V} \langle Vir \rangle \tag{26}$$

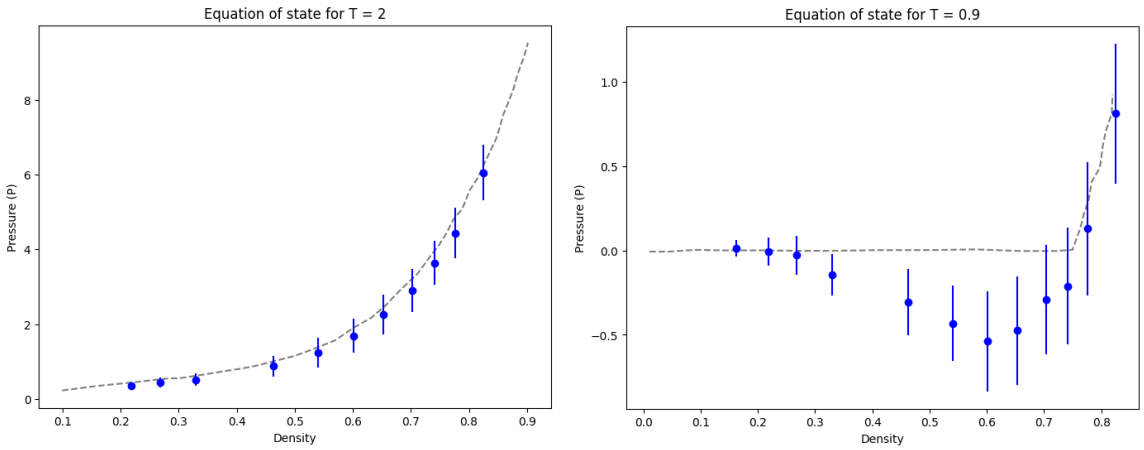

Figure 30: Equation of state in (P, ρ) plane of a LJ fluid of 100 particles at temperature above the critical one (left) and below (right) (blue dots). Grey dashed line are theoretical values.

Final pressure is calculated by summing to P_{eq} the tail correction u_P. Finally, $P = P_{eq} + u_P$ and it is represented in the plane (P, ρ) for $T^* = 0.9$ and $T^* = 2$. Density is changed by changing the size of the box [4.95, 5.05, 5.13, 5.22, 5.35, 5.5, 5.7, 6, 6.72, 7.2, 7.7, 8.5].

9 Lecture 9: Integration schemes

For the harmonic oscillator in 1 dimension, the equations of motion are:

$$\begin{cases} \frac{dq}{dt} = p \\ \frac{dp}{dt} = -\omega^2 q \end{cases} \tag{27}$$

The analytical solution, having as initial conditions $(q_0, p_0) = (1, 0)$.

$$\begin{cases} q(t) = \cos(\omega t) \\ p(t) = -\omega \sin(\omega t) \end{cases} \tag{28}$$

Now, to integrate numerically the equations of motion 27 two algorithms have been applied: Velocity Verlet algorithm and Predictor-Corrector.
The first one is quite accurate and preserves energy conservation. It was implemented as follows, assuming also $\omega = 1$ and an integration step of $dt = 0.01$, which ensures the stability of the algorithm. The algorithm was repeated for $T = 1000/0.01 = 10^5$ times. Starting from positions and momentum and the force at a given time t, $f(t) = \frac{dp}{dt} = -\omega^2 q(t)$, the momentum is shifted by $\rightarrow p_{half} = p + \frac{1}{2}(-\omega^2 q)\Delta t$, then the positions is updated as $q(t + \Delta t) = q(t) + p_{half}\Delta t$, then, the new force can be calculated $f_{t+\Delta t} = -\omega^2 q(t + \Delta t)$ and so the new momentum $p(t + \Delta t) = p_{half} + \frac{1}{2} f(t + \Delta t)\Delta t$

```
def VelVerlet(omega, dt):
    q1 = np.zeros(steps)
    p1 = np.zeros(steps)
    t1 = np.zeros(steps)
    energy1 = np.zeros(steps)
```

```
# initial conditions
    q1[0] = q0
    p1[0] = p0
    energy1[0] = 0.5*p0**2 + 0.5*omega**2*q0**2

    for n in range(steps - 1):
    # current acceleration
        a_n = -omega**2 * q1[n]
    # half kick
        p1_half = p1[n] + 0.5 * a_n * dt

        q1[n + 1] = q1[n] + p1_half * dt

        a_next = -omega**2 * q1[n + 1]

        p1[n + 1] = p1_half + 0.5*a_next*dt

    # energy
        energy1[n + 1] = 0.5 * p1[n +1]**2 + 0.5 * q1[n + 1]**2 * omega**2

    # updated time
        t1[n + 1] = t1[n] + dt
    return q1, p1, t1, energy1
```

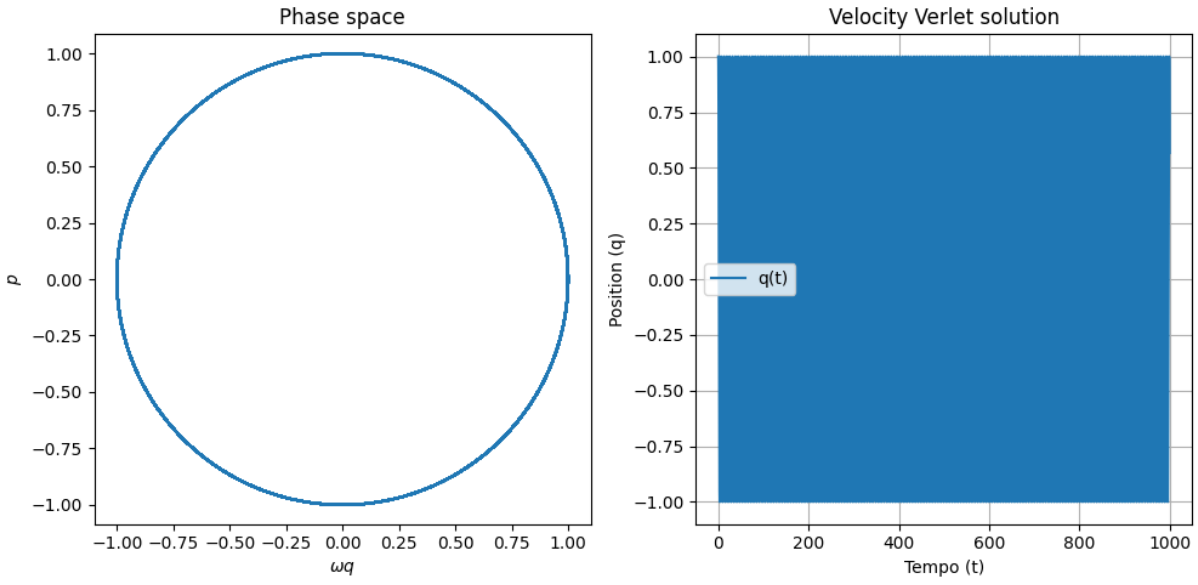

Figure 31: Phase space (left) and coresponding solution from Velocity Verlet algorithm (right), dt = 0.01, $\omega = 1$

Regarding Predictor-corrector algorithm, at first position and its derivative at a certain

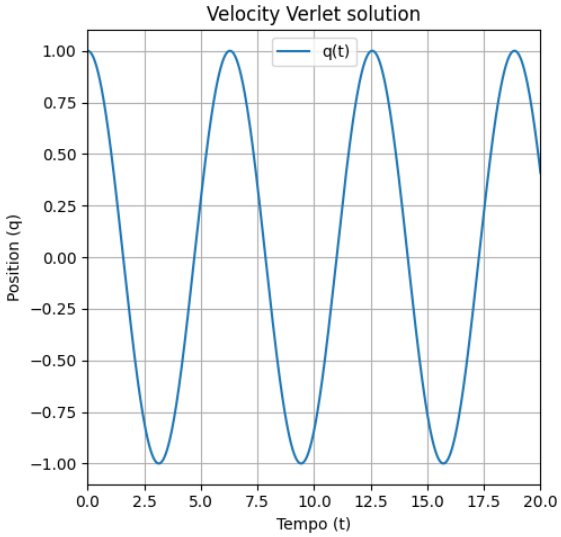

Figure 32: Solution of Velocity Verlet in a smaller region. Cosine function is clearly observable

time are calculated:

$$\begin{cases} x_1 = \frac{dq}{dt}\Delta = p\Delta t \\ x_2 = \frac{d}{dt}(\frac{dq}{dt})\frac{\Delta t^2}{2!} = -\omega^2 q\frac{\Delta t^2}{2!} \\ x_3 = \frac{d^3q}{dt^3}\frac{\Delta t^3}{3!} = -\omega^2\frac{dq}{dt}\frac{\Delta t^3}{3!} = -\omega^2 p\frac{\Delta t^3}{6} \end{cases} \tag{29}$$

Then, predicted values are:

$$\begin{cases} q^{pred}(t+\Delta t) = q(t) + x_1 + x_2 + x_3 \\ x_1^{pred}(t+\Delta t) = x_1(t) + 2x_2(t) + 3x_3(t) \rightarrow p^{pred}(t+\Delta t) = \frac{x_1(t)+2x_2(t)+3x_3(t)}{\Delta t} \\ x_2^{pred}(t+\Delta t) = x_2(t) + 3x_3(t) \end{cases} \tag{30}$$

the "corrected" $x_2(t)$ can be calculated as $x_2(t)^{corr} = -\omega^2 q^{pred}(t+\Delta t)\frac{dt^2}{2}$, and this will be different from $x_2^{pred}(t+\Delta t)$ in 30, so a difference $\Delta x_2 = x_2^{corr} - x_2^{pred}$ can be computed and used to calculate the other corrections:

$$x_n^{corr} = x_n^{pred} + C_n\Delta x_2 \tag{31}$$

with C_n being some tabulated values. In this case: $C_0 = 1/6$, $C_1 = 5/6$, $C_2 = 1$, $C_3 = 1/3$. Therefore $q^{corr}(t+dt) = q^{pred} + (1/6)\Delta x_2$ and $p^{corr}(t+dt) = p^{pred}(t+dt) + C_1\Delta x_2/dt$. Here the function used in Python is reported:

```
def PredictorCorr(omega, dt):
    q2 = np.zeros(steps)
    p2 = np.zeros(steps)
    t2 = np.zeros(steps)
    energy2 = np.zeros(steps)
    q2[0] = q0
    p2[0] = p0
    energy2[0] = 0.5 * p0**2 + 0.5 * omega**2 * q0**2
    for n in range(steps - 1):
```

```
        # definitions
        x_1 = p2[n] * dt
        x_2 = (1 / 2) * (-q2[n] * omega**2) * dt**2
        x_3 = (1 / 6) * (-omega**2) * dt**3

        q_pred = q2[n] + x_1 + x_2 + x_3
        x_1_pred = x_1 + 2 * x_2 + 3 * x_3
        x_2_pred = x_2 + 3 * x_3
        x_3_pred = x_3
        x_2_corrected = (1 / 2) * (-q_pred * omega**2) * dt**2
        delta_x2 = x_2_corrected - x_2_pred
        q2[n + 1] = q_pred + (1 / 6) * delta_x2
        p2[n + 1] = (x_1_pred + (5 / 6) * delta_x2) / dt   # p = x_1/dt

        energy2[n + 1] = 0.5 * p2[n + 1]**2 + 0.5 * omega**2 * q2[n + 1]**2
        t2[n + 1] = t2[n] + dt
    return q2, p2, t2, energy2
```

Results in the phase space and in the solution $q(t)$ from the application of this integrator with $dt = 0.01$ and $\omega = 1$,as in the previous case, are shown in figure 33 clearly show the instability of the algorithm, at this order of approximation and for this integration step. To improve the precision of the integrator, one should go further in the Taylor expansion and / or diminish the integration step dt.

Here in figure 34 the deviation $q^{numerical}(t) - q^{analytical}(t)$ is represented. Looking now

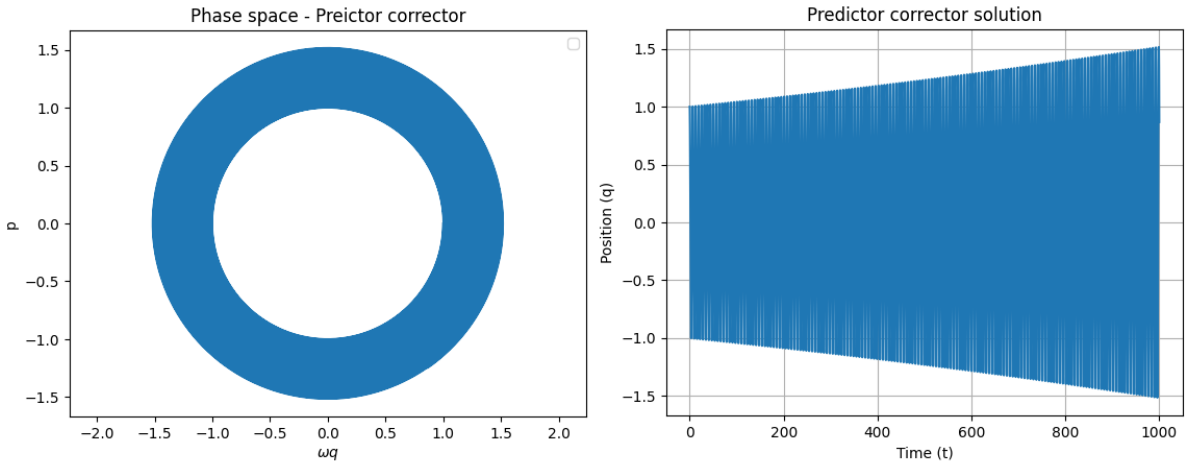

Figure 33: Phase space (left) and solution of Predictor-corrector algorithm. $\omega = 1$ and $dt = 0.01$. Deviation from the expected solution is evident

at the total energy, kinetic plus potential, and how it is preserved in time, in figure 35 it can be seen that with Predictor-corrector algorithm the conservation is completely lost during the integration. Velocity Verlet so far seems very accurate, however the variation of the product ωdt, dt being the integration time step, can result in cumulative errors which cause the numerical solution to deviate increasingly from the analytical one. Here in figure 36

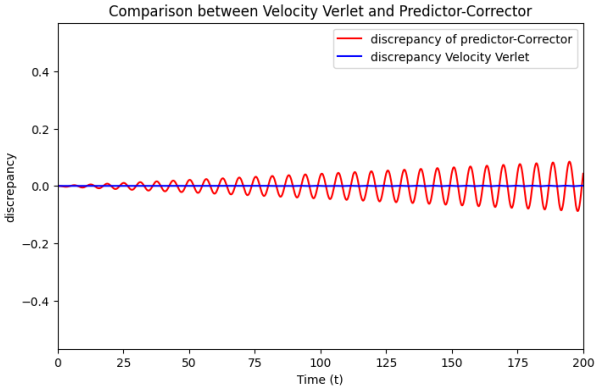

Figure 34: difference between numerical solution of the harmonic oscillator, calculated both with Velocity Verlet and with Predictor-corrector algorithm, and the analytical solution

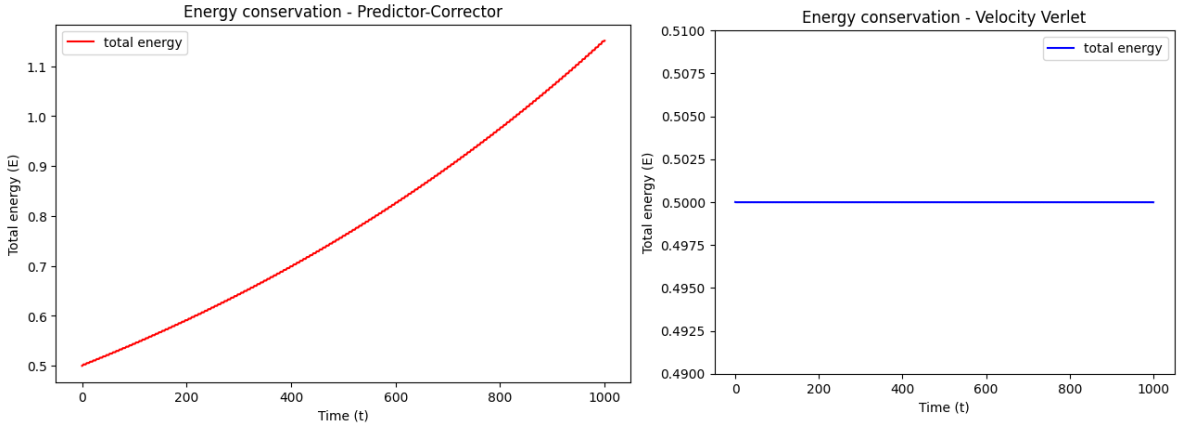

Figure 35: Total energy from Predictor-corrector algorithm (left) and from Velocity Verlet (right)

10 Lecture 10: Thermostats

Canonical fluctuations

Kinetic temperature is defined as

$$T_K = \frac{1}{3Nk_B}\sum_i m_i v_i^2 \tag{32}$$

The relationship with the kinetic energy is then

$$T_K = \frac{2}{3Nk_B}E_K \tag{33}$$

To get the variance of E_K and then of T_K one can evoke the equipartition theorem, according which

$$\langle E_K \rangle = \frac{3N}{2\beta} \tag{34}$$

Therefore

$$\sigma^2_{E_K} = -\frac{\partial}{\partial\beta}\langle E_K \rangle = \frac{3N}{\beta^2} \tag{35}$$

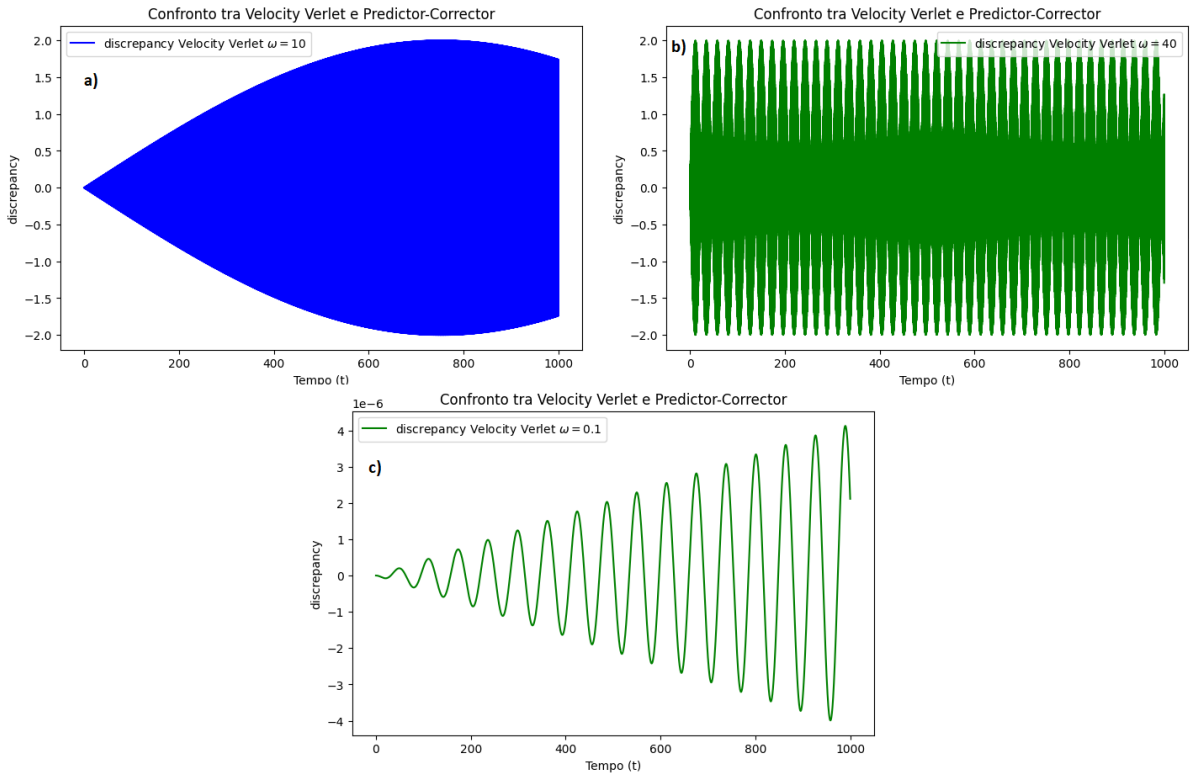

Figure 36: Deviations varying ω keeping $dt = 0.01$ fixed. a) $\omega = 10$; b) $\omega = 40$; c) $\omega = 0.1$

Putting together the last result with the previous ones one gets

$$\frac{\sigma_K^2}{\langle T_K \rangle^2} = \frac{2}{3Nk_B^2\beta^2} \frac{9N^2k_B^2}{4\langle E_K \rangle^2} = \frac{3N4\beta^2}{2\beta^2 9N^2} = \frac{2}{3N} \tag{36}$$

Lennard - Jones fluid in different ensembles

NVE ensemble

In this MD simulation, initial positions of the fluid are generated using the MC simulation of exercise 8 for 2000 time steps, to produce a configuration of equilibrium. This is done both for NVE and NVT ensemble. Initial velocities, as suggested, are generated according to Maxwell Boltzmann distribution at $T^* = 1$. From these inital velocities and positions, equations of motion are integrated with Velocity Verlet algorithm. This process is repeated for different cutoff radii of the potential (with the potential shifted each time to ensure continuity, which affects both the efficiency of the algorithm and then energy conservation). The forces on particle i are calculated by first computing $\vec{r_{ij}} = \vec{r_i} - \vec{r_j}$. Then $\vec{f_{ij}} = -\frac{dV}{dr}\frac{\vec{r_{i,j}}}{|r_{ij}|}$ and of course $\vec{f_{ji}} = -\vec{f_{ij}}$. Then $\vec{F_i} = \sum_j \vec{f_{ij}}$. The total energy is computed at each time step to monitor energy conservation. Time step of the algorithm is $dt = 0.001$. 5 radial distribution function are calculated, each separated by 1000 repetitions of the algorithm and then an average of them is computed.
In the final plot, figure 37, some oscillations remain; this suggests that larger sample of radial distribution functions should be needed to improve the result. For a small cutoff radius, the RDF is compatible with a gas phase, with a low peak located approximately at the equilibrium distance of the LJ potential ($r_{eq} = 2^{1/6}\sigma$); at higher cutoff radii, par-

ticles begin to feel attraction leading to the formation of a second peak in the RDF. For $r < r_{eq}$ the RDF goes rapidly to zero , due to the highly repulsive part of the LJ potential which prevent the particles to be at so short distances from any reference particle.

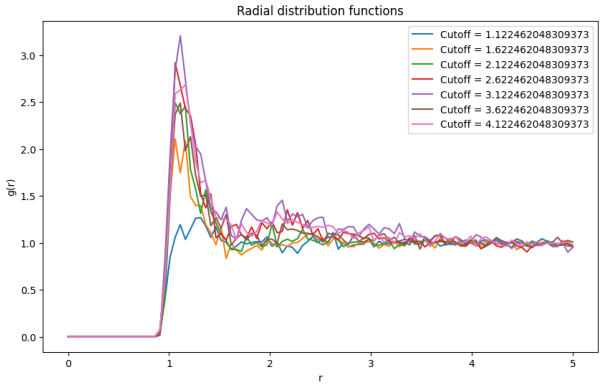

Figure 37: Average, for different cutoff radii, of 5 radial distribution functions separated each by 1000 integration steps.

NVT ensemble

Now the system is put in contact with a heat bath of reduced fixed temperature $T = 2$, hence now the system is simulated in a canonical ensemble, in which now the energy does not conserve anymore. Here two thermostats are introduced: Velocity rescaling thermostat and Andersen thermostat. The procedure is the same of the NVE simulation: first there is the equilibration with the MC simulation and the integration with Velocity Verlet algorithm. But now for the first thermostat, velocity rescaling, at each integration step velocities are rescaled by $\lambda = \sqrt{\frac{2}{T_K(t)}}$. Instead for Andersen thermostat, the velocity of randomly chosen particles is rewritten, with new values drawn from the equilibrium Maxwell–Boltzmann distribution to simulate collisions with a heat bath. These collisions occur at a rate ω; at each time step, a particle's velocity is reset with probability ωdt.
Integration with thermostats is carried out with simulation time of 2000 steps. The simulation is repeated with different number of particles. Mean kinetic energy is calculated each time to verify the theoretical expectation in the canonical ensemble: $\langle E_K \rangle = \frac{3}{2} N k_B T = 3N$ ($k_B = 1$). Also, fluctuations of the temperature are calculated to test the prediction: $\frac{\sigma^2_{T_K}}{\langle T_K \rangle^2} = \frac{2}{3N}$. Results are reported in the following table 2. As expected, the fluctuations of the temperature relative to Velocity rescaling thermostat do not follow the canonical ensemble prediction, whereas Andersen does. The average kinetic energy for both thermostats is in line with expectation, although being more accurate for Velocity rescaling. In figure 38 thermal fluctuations resulting from both thermostats are reported.

N	$\langle E_k^{theor} \rangle$	$\langle E_k^{V.resc} \rangle$	$\langle E_k^{Anders} \rangle$
100	300	300.0 ± 1.9	301 ± 25
140	420	420 ± 3	420 ± 29
170	510	510 ± 4	503 ± 32
200	600	600 ± 4	598 ± 36

Table 2: Mean kinetic energy comparison among Andersen results, Velocity rescaling result and theoretical result

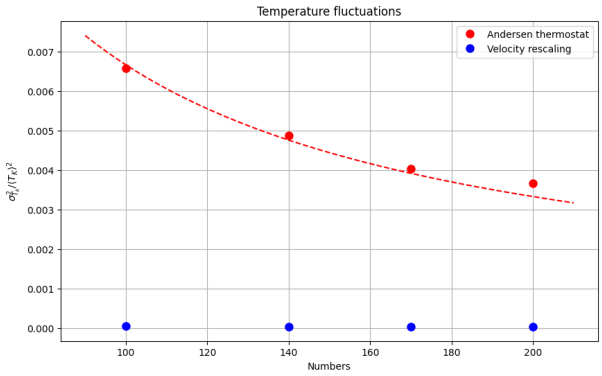

Figure 38: Temperature fluctuations in canonical ensemble. Red dots: using Andersen thermostat. Blue dots: with velocity rescaling thermostat.

11 Lecture 11: Langevin and Brownian dynamics

Brownian time and length scales

1) Considering a spherical particle of diameter $\sigma = 10^{-8}m$ diffusing in water over a distance equal to its diameter and knowing that the correlation time of water molecules is of the order of picoseconds, taken as the time scale of the fluctuation, so $\tau_{coll} = 10^{-12}s$, we have that the diffusive behavior is described by a relation like $\langle (x(t) - x_0)^2 \rangle \approx 2Dt$ with D being the diffusion coefficient which can be estimated with the Einstein-Stokes relation as $D = \frac{k_B T}{6\pi\eta(\sigma/2)}$. For water at T $\sim$ 300K, the viscosity $\eta = 8.9 \cdot 10^{-4} Pa \cdot s \sim 10^{-3} Pa \cdot s$; $k_B = 1.28 \cdot 10^{-23} \mathrm{JK}^{-1}$. Therefore:

$$t^{(1)}_{scale} \approx \frac{\sigma^2}{2D} = \frac{3\sigma^3 \pi \eta}{2k_B T} \sim 10^{-6}s >> \tau_{coll} \qquad \text{with } \sigma = 10^{-8}m \tag{37}$$

The timescale of the diffusion is much higher the characteristic time of the collisions in water, showing that the Brownian motion description is valid.

2) When $\sigma \geq 5\mu m$, the corresponding diffusion time scale is then

$$t^{(2)}_{scale} \approx 1s \qquad \text{with } \sigma = 5\mu m \tag{38}$$

which again is much higher than the time scale of the collisions. However, with such larger particle, gravitational force becomes relevant. Thermal energy is $k_B T \approx 4 \cdot 10^{-21} J$ at T=300K, which is the energy scale of the diffusion. While gravitational energy can be estimated as $U = \rho V g h$. If $h = \sigma$ and $V = \frac{4}{3} g R^3 = \frac{4}{3} g (\frac{\sigma}{2})^3$ and $\rho \sim 1000\,\mathrm{kg/m^3}$, then $U(\sigma = 10^{-8}m) \sim 10^{-29} J$ and $U(\sigma = 5 \cdot 10^{-6}m) \sim 10^{-18} J$. Comparing them with the estimation above of the thermal energy, one sees immediately that in the second case the gravitational energy becomes more significant than the thermal one, meaning that Brownian motion becomes negligible. Another way to see it is to compare the time scales calculated above with the one relative to the gravitational force. Indeed, if naively $F = mg$ and $\Delta x = \frac{1}{2} g t^2$, then $t^g_{scale} = \sqrt{\frac{2\Delta x}{g}} = \sqrt{\frac{2\sigma}{g}}$. Then $t^g_{scale}(\sigma = 10^{-8}m) = 4.5 \cdot 10^{-5}s > t^{(1)}_{scale}$ and $t^g_{scale}(\sigma = 5 \cdot 10^{-6}m) \approx 10^{-3}s < t^{(2)}_{scale}$. Diffusion in the first result is faster than the action of gravitation, being then the relevant motion; while in the second result, diffusion is much slower than the dynamical effect of the gravitational force.

Brownian motion:
1) Simulation is done with 600 particles for a total simulation time of 10^4 steps. For the ideal gas case, the gas is simulated in the overdamped regime in 2 dimensions, thus integrating Langevin equations using a first order integrator as:

$$x(t+dt)_i = x(t)_i + \sqrt{\frac{2k_BT}{m\gamma}}\xi_{i,x}\sqrt{dt} \tag{39}$$

$$y(t+dt)_i = y(t)_i + \sqrt{\frac{2k_BT}{m\gamma}}\xi_{i,y}\sqrt{dt} \qquad i = 1, ..., N_{particles} \tag{40}$$

The integration is done numerically with the following function, having a step dt = 0.01. Initial positions are set in the middle of the 2-dimensional of size L $20\sigma = 20$. Periodic boundary conditions are imposed.

```
    def overdamped_interacting(T, g, K):
# multidimensional array in which, for each particle,
# positions in xy plane are stored and at each time.
    positions = np.zeros((N_particles, 2, steps))
    nopbc_positions = np.zeros((N_particles, 2, steps))
    positions[:, :, 0] = L / 2
    nopbc_positions[:, :, 0] = L / 2

    for t in range(1, steps):
        # gaussian noise
        xi = np.random.normal(0, 1, size=(N_particles, 2))
        # first order integrator
        interaction = - (K / (m * g)) * (positions[:, :, t-1] -
        positions[:, :, 0]) * dt
        positions[:, :, t] = positions[:, :, t-1] + interaction +
        np.sqrt(2 * k_B * T / (m * g)) * xi * np.sqrt(dt)
        nopbc_positions[:, :, t] = positions[:, :,t]
        # PBC
        positions[:, :, t] = positions[:, :, t] -
        np.floor(positions[:, :, t] / L) * L

    return nopbc_positions
```

The mean square displacement is the calculated using unwrapped coordinates as:

```
    def msd(positions):
    delta = positions - positions[:, :, np.newaxis,0]  # r(t) - r(0)
    msd_values = np.mean(np.sum(delta**2, axis=1), axis=0)

    return msd_values, delta
```

In the following image the mean square displacement is plotted against time, with different friction coefficients γ, starting from $\gamma = 10$ in order to have a ratio $m/\gamma = 1/\gamma$ small

enough to be compatible with an overdamped regime, in which inertial effects are negligible. Temperature is fixed at $T^* = 1$ As it can be observed, the mean square displacement grows linearly with respect to time evolution. This is consistent with a Brownian motion mean square displacement $\Delta r = 2Dt$. The slope grows while γ decreases, in line with Einstein relation $\frac{k_B T}{m\gamma} = D$. Estimation of D coefficient is then done from the slope b of the linear fit $a + bx$: $D = b/2$. Also, the same procedure is repeated fixing $\gamma = 10$ and changing temperature $0.1 < T^* < 2$. It can be observed immediately that the slope now increases with temperature, again consistently with Einstein relation.

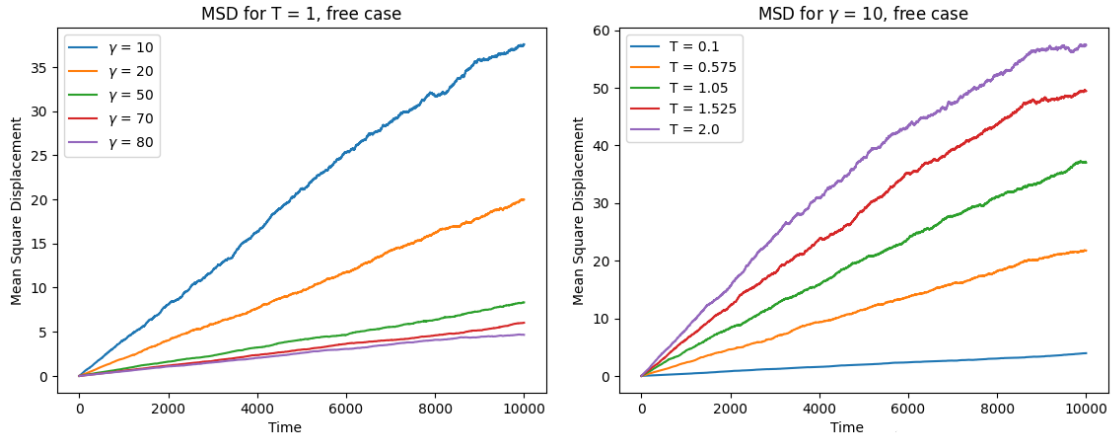

Figure 39: Mean square displacement of free overdamped Langevin dynamics. Right panel: Fixed $\gamma = 10$ and variable T^*. Left panel: fixed reduced temperature $T^* = 1$ and changing γ.

T_i	D($\gamma = 10$)	D_{Δ_x}	γ_i	D($T^* = 1$)	D_{Δ_x}
0.1	0.0002	0.0001	10	0.002	0.001
0.575	0.001	0.0005	20	0.001	0.0005
1.05	0.002	0.001	50	0.0004	0.0002
1.525	0.003	0.001	70	0.0003	0.0001
2	0.003	0.002	80	0.0002	0.0001

Table 3: Diffusion coefficient from linear fit on msd, with either changing temperature or γ. Estimates from the gaussian fit on the distribution of the displacements along x is reported too. Clearly there is a factor of 2

The distribution now of the displacement $x_i - x_0$ along one axis, for example along x axis, is plotted at different times, in this case at $t = [1, 10, 100, 1000, 8000]$. One can see that the distribution, as one expects from a Brownian motion, is compatible with a Gaussian one. Additionally, as t increases, the variance increases and the particles spread more. Here an example is reported. A Gaussian fit is performed and then the variance σ is retrieved at a given time. From the variances at different times, another estimation of D is done: since a behavior like $\sigma(t) \sim \sqrt{t}$ is expected, an interpolation with a function $f(t) = a\sqrt{t}$ is done; then, from the resulting parameter one has $a = \sqrt{2D}$. Hence $D = a^2/2$. In table 11 the diffusion coefficients estimated from the distribution of the displacement along x are reported.

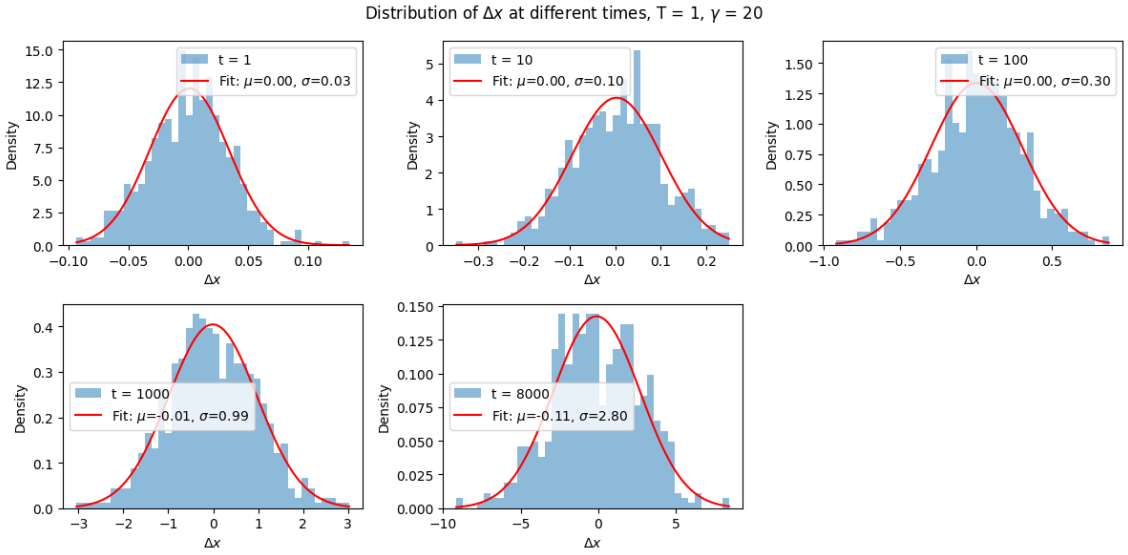

Figure 40: gaussian distribution in non interacting case at different times

2) **Brownian motion in harmonic trap**

With a harmonic trap now, the particles are not free to diffuse in the whole space; this behavior should be visible in the mean square displacement. In figure 41 it is reported the MSD with different γ and T as done in the non-interacting case. Additionally, the same quantity has been calculated changing also the strength of the harmonic confinement, namely K. It can be seen that, as expected, the MSD grows linearly like in the free case just for a limited period of time, starting then to oscillate around a saturation value. If the temperature grows, the saturation value of the Msd grows as well, since particles have more thermal energy to explore a bigger region in the harmonic trap. Now, varying, the

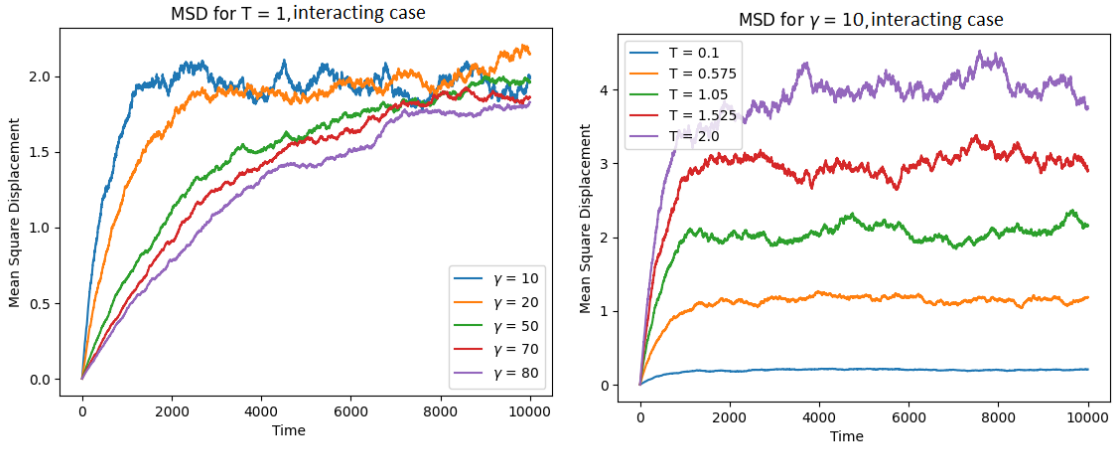

Figure 41: mean square displacement in the interacting case. K is fixed in both cases, varying either γ or the temperature

constant K of the harmonic trap, one can notice in figure 42 that the saturation value of the MSD gets lower as K increases. This is expected as the harmonic trap, becoming stronger, confine the particles even more, limiting the diffusion. To conclude, it is interesting noticing that the displacement on x axis with the harmonic trap has a different distribution with respect to the non interacting case. Here in figure 43, the distribution

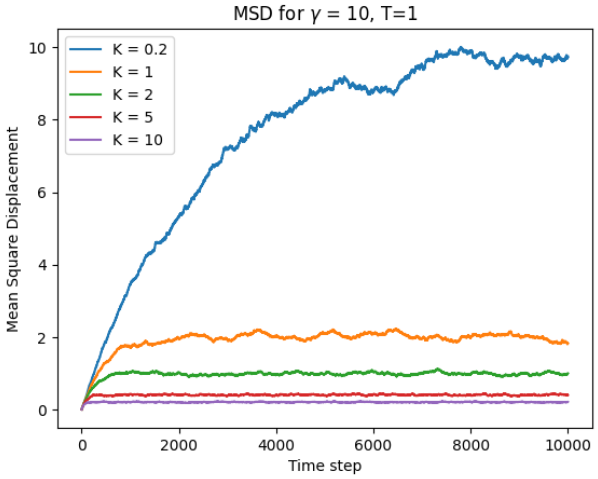

Figure 42: Msd in interacting case with different values of K. As K grows, particles are more and more confined.

of the displacement along x clearly shows the effect of the harmonic trap: while time increases, the distribution at a certain point does not spread anymore, with a standard deviation that is almost constant. This is in line with the expectation and also with the behavior of the mean square displacement shown above. The harmonic confinement opposes to the diffusion.

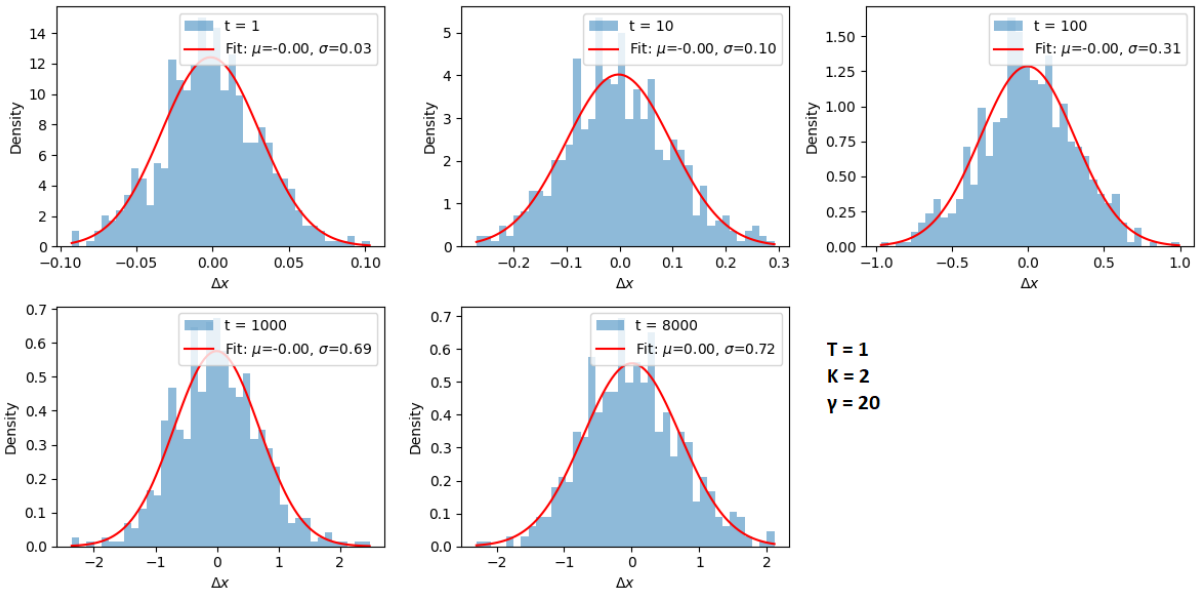

Figure 43: distribution of the displacement on x at different time, with interaction. Effects of confinement are visible since the spread of the distribution stops growing

12 Lecture 12: Multiple histogram method

The Multiple histogram method has been implemented for the grafted polymer simulated in the exercise 6 to estimate the variance of the energy per monomer as a function of the

temperature $C(T) = \frac{\langle E^2\rangle - \langle E\rangle^2}{N}$ and to estimate the critical temperature at the thermodynamic limit.
First step is the self consistent determination of the partition function, starting from an initial guess $\vec{Z} = (Z_0, ..., Z_{k-1}) = (1, ..., 1)$, having sampled the energy of the polymer at different k temperatures $T_0, ..., T_{k-1}$.

$$Z_i = \sum_E \frac{\sum_j N_j(E)}{e^{\lambda_{i0}} \sum_m e^{\lambda_{im}(E) - \lambda_{i0}(E)}} \tag{41}$$

where the indices $i, j = 0, ..., k-1$, $\lambda_{kj}(E = \log(M_j) - \log Z_j + (\frac{1}{k_B T_k} - \frac{1}{k_B T_j})E$. $N_j(E)$ are the counts of the given energy E at temperature T_j and M_j is the total number of "measurements" of energy at temperature T_j. Equation 41 is initially calculated using the initial guess mentioned above. Then the resulting Z is used in the following iteration in place of the starting guess, and the procedure is repeated, getting more and more precise estimation of Z. The iterations stop when $\Delta^2(t) = \sum_j \left(\frac{Z_j^{(t)} - Z_j^{(t-1)}}{Z^{(t)}{}_j} \right)^2 < \epsilon^2 = 10^{-14}$.
After the full $\vec{Z}_{sc}$ is determined self consistently, it can be used to derive every thermodynamic variable at any temperature T.

$$Z(T) = \sum_E \frac{\sum_i N_i(E)}{\sum_j \frac{M_j e^{(1/k_B T - 1/k_B T_j)E}}{Z_{sc,j}}} \tag{42}$$

$$\langle O\rangle_T = \frac{1}{Z_T} \sum_E O(E) \frac{\sum_i N_i(E)}{\sum_j \frac{M_j e^{(1/k_B T - 1/k_B T_j)E}}{Z_{sc,j}}} \tag{43}$$

Therefore $C(T)$ for a fine interval of temperatures is estimated employing 42 and then 43. The whole procedure is repeated for different number of monomers: [10, 15, 25]. An important note: the interval of sampled temperatures for each N is chosen in order to ensure a sufficiently high swapping rate between temperatures, in order to avoid any possible bottleneck regions which affect the result sensibly. Results are reported in figure 44. As it can be expected, fluctuations of the energy are more sensible to hypothetical inaccuracies. From the curves of $C(T)_N$ the critical temperature at that N is estimated as the value of temperature that gives the maximum fluctuation. The results are:

$$T_{10} \approx 0.54 \qquad T_{15} \approx 0.58 \qquad T_{25} \approx 0.6 \tag{44}$$

These results are used to estimate at the thermodynamic limit $N \to \infty$. More in detail: the critical temperatures are plotted as a function of $N^{-\phi}$, ϕ being the crossover exponent; in this case $\phi = 1/2$ guarantees a decent alignment of the points (even though are very few, there would be the need to increment the number estimates at different N). Then a linear fit is done $a + bx$, so that a gives an estimation of the T_c for $N \to \infty$. Final result is:

$$T_c = 0.71 \pm 0.03 \tag{45}$$

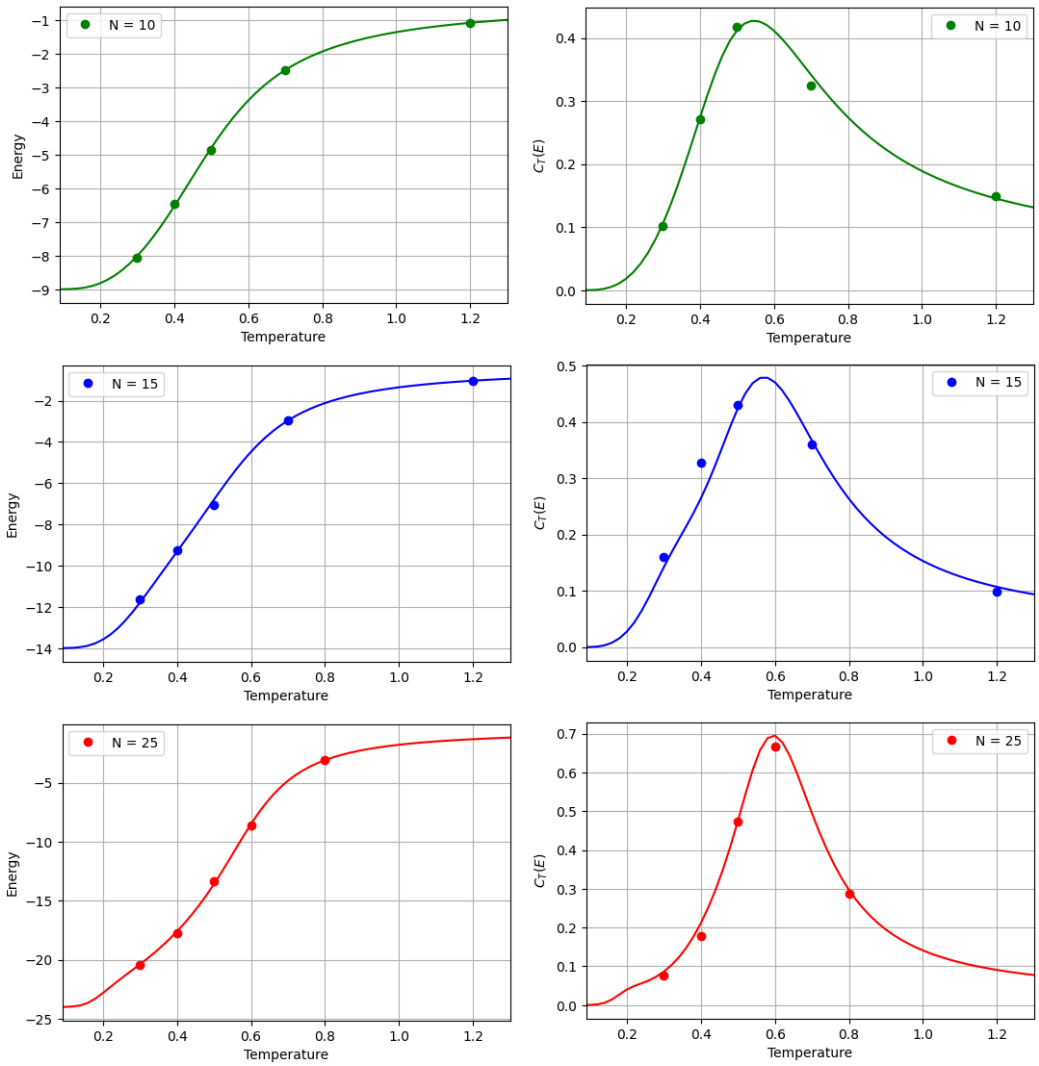

Figure 44: Results from application of Multiple Histogram Method with different polymer lengths

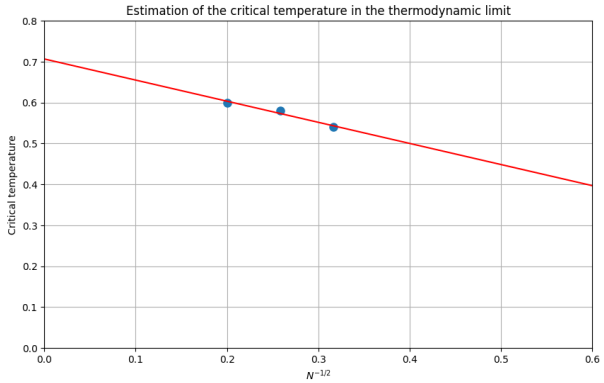

Figure 45: Linear fit of the critical temperatures as function of $N^{-1/2}$. From the intersection with y axis, T_c at $N \to \infty$ is found.

13 Lecture 13: Cell-list

In this simulation, particles are simulated in a box of size B = 100 with reflective boundary conditions. The first half of them has radius $R_A = 1$ and the second half $R_B = 1.25$. They are subjected to a potential $U(r_{ij}) = \epsilon e^{-r_{ij}^2/(2\sigma_{ij}^2)}$. The size of cell l is chosen as the maximum cutoff, namely $4(1.25 + 1.25) = 10$. In this way, if a particles belongs to a given cell, the particles that can interact with it can be searched only in the nearest neighbor cells, which is a convenient procedure. To find the list of particles that at a given time occupy a given cell which has label, or index, c, the algorithm uses three arrays: a tag list τ which has N_c elements, with N_c being the number of cells, that keeps track of whether the cell is occupied during the current simulation step T_{cell}. If $\tau(c)! = T_{cell}$, cell c is considered empty; a list of head pointer ϕ with length N_c: this list stores the index of the first particle that occupies the cell. From this index, one can reconstruct the indices of the particles that are currently in that cell by emplying the last list: the particle list ν, which has length N number of particles, creates a linked list of particles in each cell. For a given particle $i, \nu(i)$ holds the index of the next particle in the same cell (or a special value like -1 if i is the last particle in that cell. At each integration time step, these lists are rest to their starting values. For every particle i, the cell index is calculated. If $\tau(c)$ does not match the current time-step T_{cell}, $\phi(c) = i$, $\tau(c) = T_{cell}$ and $\nu(i) = -1$. If the cell is already occupied, then $\nu(i) = \phi(c)$ and $\phi(c)$ is updated $\phi(c) = i$.
After the cell list is populated, for each cell c with $\tau(c) = T_{cell}$ the forces are computed for each particle i interacting with particles j in the neighbor cells. For cells that are at the edge obviously the number of neighboring cells is different than for cells in the bulk. This is treated in the following way:

```
def compute_forces(N, num_cells, positions, radii, epsilon, tau, phi, nu, T_cell):
    forces = np.zeros((N, 2))
    Nc = num_cells**2

    for c in range(Nc): # Nc = total number of cells
        if tau[c] != T_cell:
            continue
# skip it, cycle restarts because we don't have any particles
#-> no need to compute forces

        # otherwise, find the headpointer
        i = phi[c]
        while i != -1:
            for dx in [-1, 0, 1]:
                for dy in [-1, 0, 1]:
                # first left column
                    if dx == -1 and (c % num_cells == 0):
                        continue
                # last right column
                    if dx == 1 and (c % num_cells == num_cells - 1):
                        continue
                # first row
                    if dy == -1 and (c < num_cells):
```

```
                continue
            # last row
                if dy == 1 and (c >= num_cells*(num_cells - 1)):
                    continue

                neighbor_cell = c + dx + num_cells * dy


                if tau[neighbor_cell] != T_cell:
                    continue
                # if the neighbor cell is empty, go ahead and look the others

                j = phi[neighbor_cell]
                while j != -1:
                    if i < j: # avoiding double counting
                        rij = positions[i] - positions[j]
                        r = np.sqrt(np.sum(rij**2))
                        sigma_ij = radii[i] + radii[j]
                        r_cut = 4 * sigma_ij

                        if r < r_cut:
                            A = (epsilon / (sigma_ij)**2) * np.exp(- r**2 /
                            (2 * sigma_ij**2))
                            force_val = A * (rij)

                            forces[i] += force_val  # Fij = forces on i from j
                            forces[j] -= force_val  # Fji = - Fij

                    j = nu[j]
    # go to the following particle nu[j] = the particle after j
    #-> j now becomes this particle
            i = nu[i]
    return forces
```

The integration is done via Euler Maruyanma algorithm. For the particle i:

$$\begin{cases} x_i(t+\Delta t) = x_i(t) + \mu_i F_{i,x}(t)\Delta t + \sqrt{2k_B T \mu_i \Delta t}\ \xi_{i,x} \\ y_i(t+\Delta t) = y_i(t) + \mu_i F_{i,y}(t)\Delta t + \sqrt{2k_B T \mu_i \Delta t}\ \xi_{i,y} \end{cases} \tag{46}$$

Now, to study the diffusivity as a function of the number density, the mean square displacement along the x axis is computed and plotted in log-log diagram. For $N = 50$ the final msd is the result of averaging over 10 runs of the simulation; for N = 100 the average is done on 5 runs. For N = 200 only 1 run. Integration time step is $dt = 0.001$ and the simulation time is $10^5 \cdot dt = 10^2$. As one can notice in figure 46, asymptotically, as N increases, the slope of the msd is reduced, indicating a smaller diffusion of the particles. This is something that is expected, since, for a more dense system, the interactions among particles start to become more significant , limiting the free diffusion of them. Considering now that particles of type B have a larger radius, one can expect that the diffusion for those kind of particles is smaller, which would result in a smaller slope of the mean square displacement.

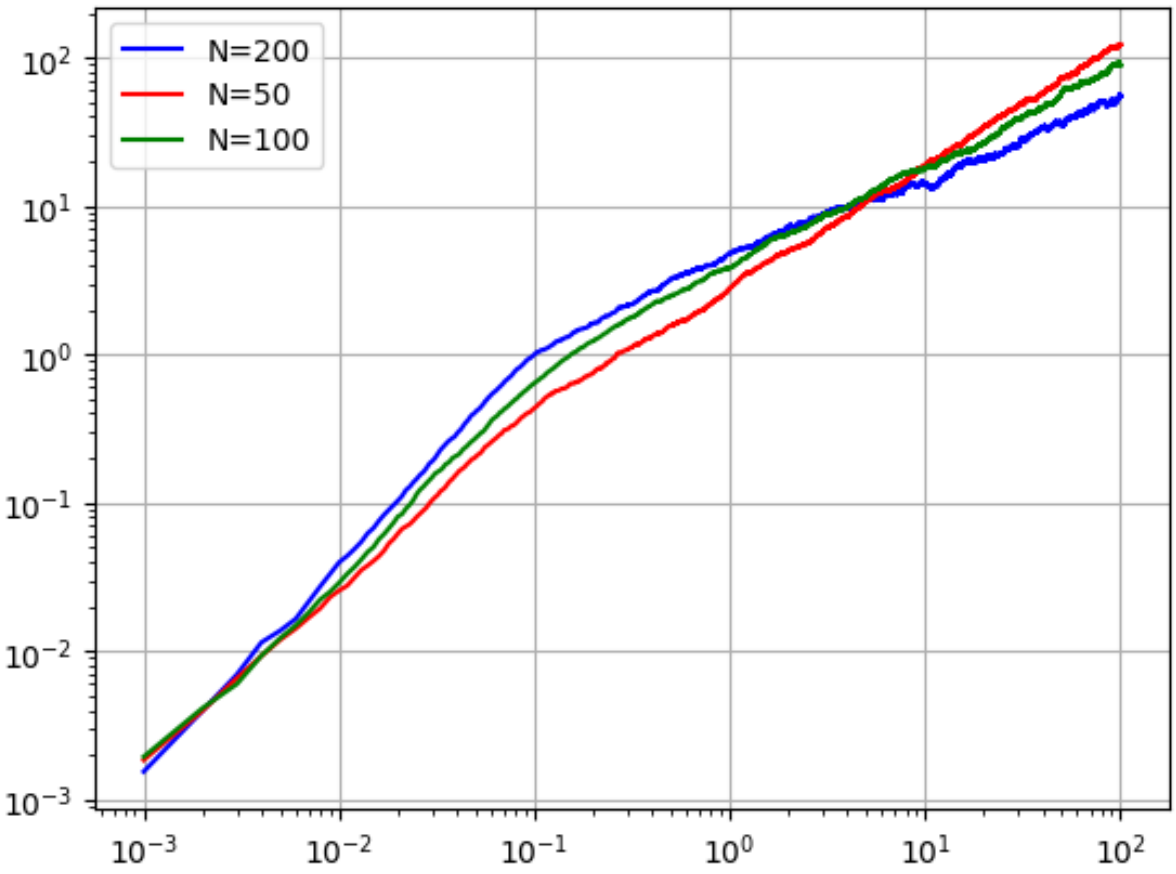

Figure 46: mean square displacement for different number of particles N